Network Administration Lab Report

Linux command evidence and Network Administrator Agent

Anas Dawod

Student ID: 12442654

Environment: Ubuntu 24.04 LTS on VMware Workstation

Part 1: Linux terminal command screenshots

Part 2: Python-based network diagnostic agent

Evidence: user-supplied screenshots and Python source file

Note: Images are preserved as supplied; screenshots include command preparation and troubleshooting.

The cal command was unavailable in the documented session and should not be marked successful.

Part 1 | Linux Command Evidence

Screenshots arranged chronologically by original capture filename

The uploaded ZIP contains 37 images, including terminal commands, setup, and troubleshooting.

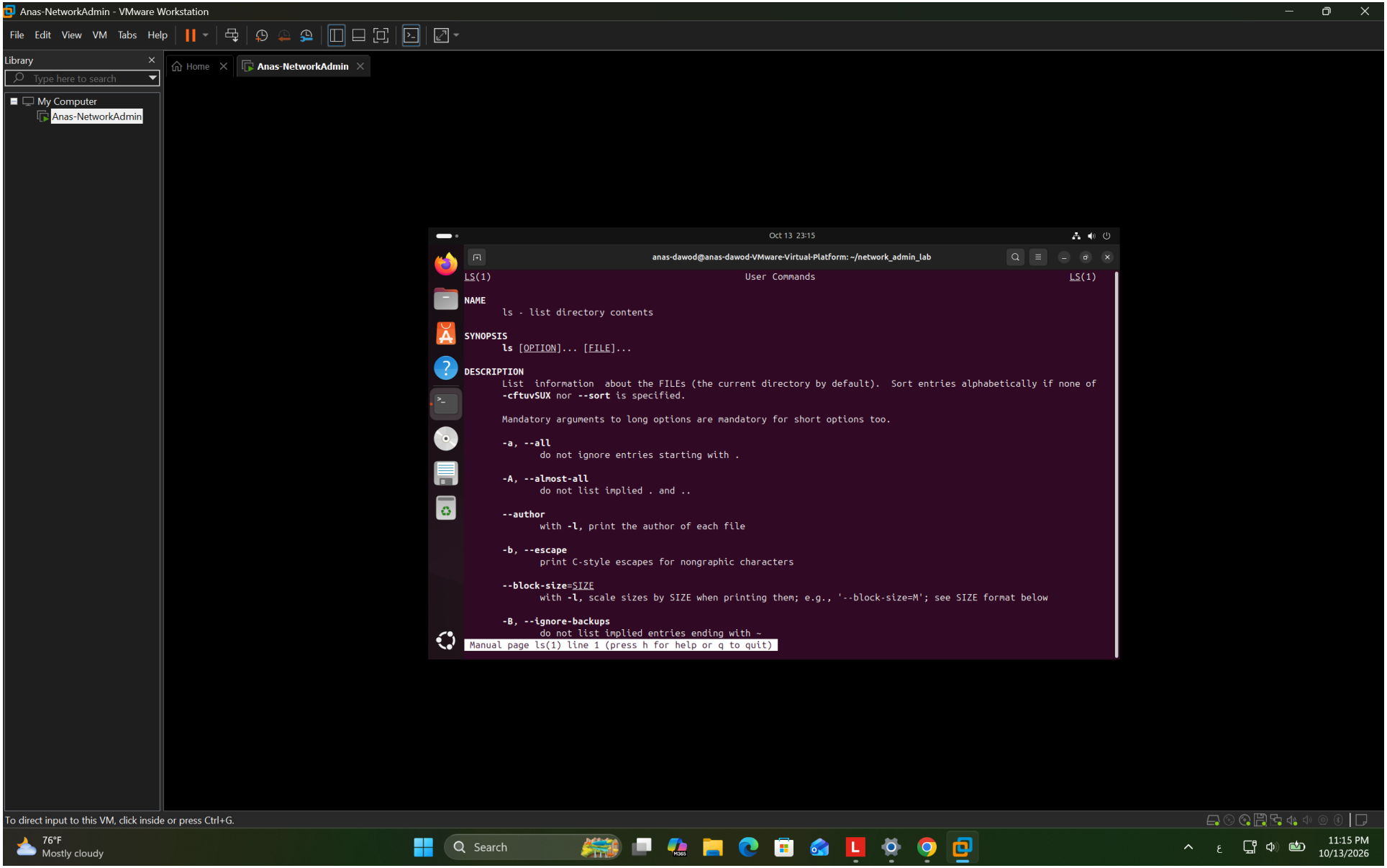
Each screenshot is presented individually without altering its terminal output.

The command-history screenshot and cal error are retained as actual evidence.

Some screenshots document supporting actions, not separate graded commands.

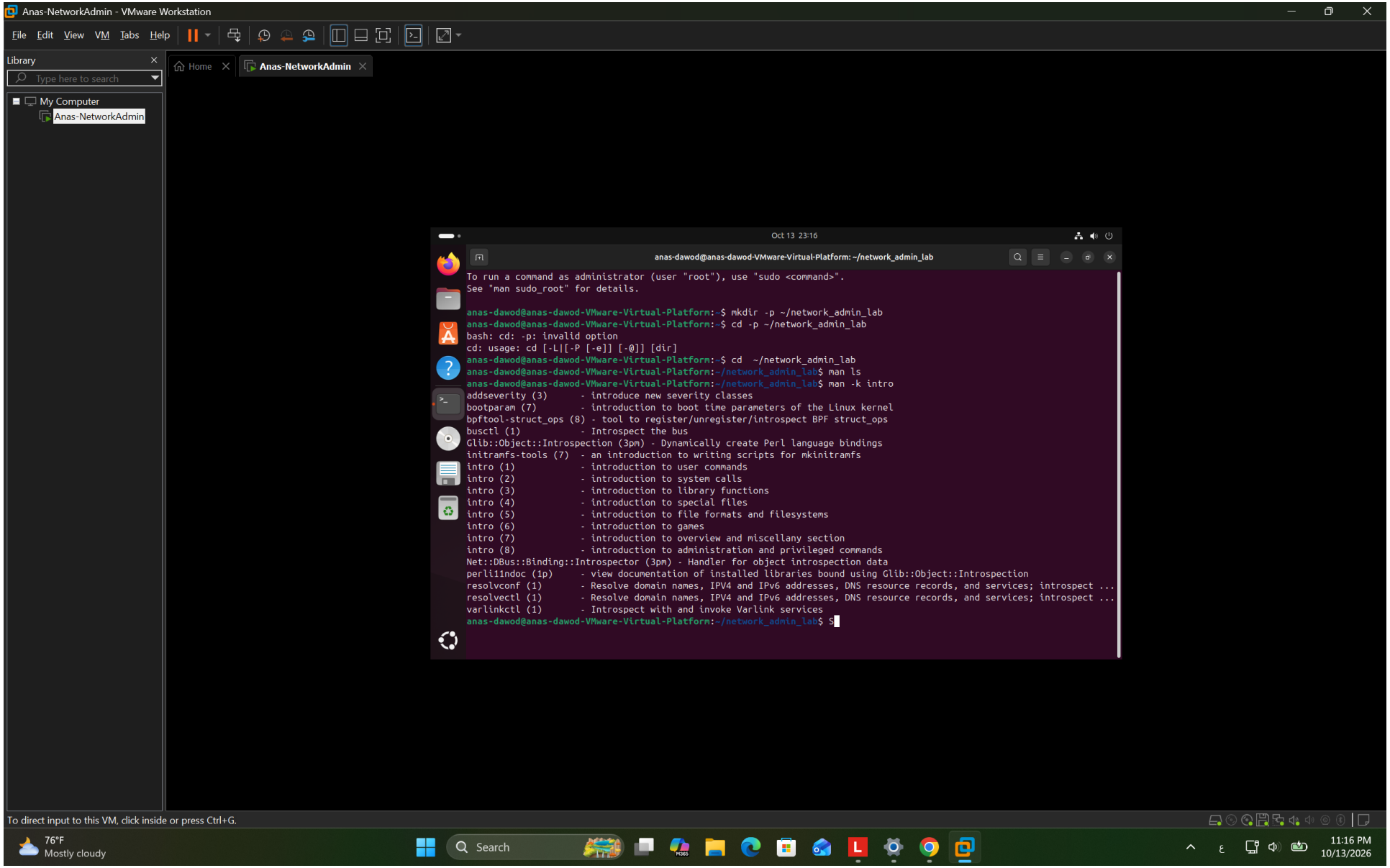
Linux Terminal Evidence 01 / 37

Screenshot 2026-10-13 231521



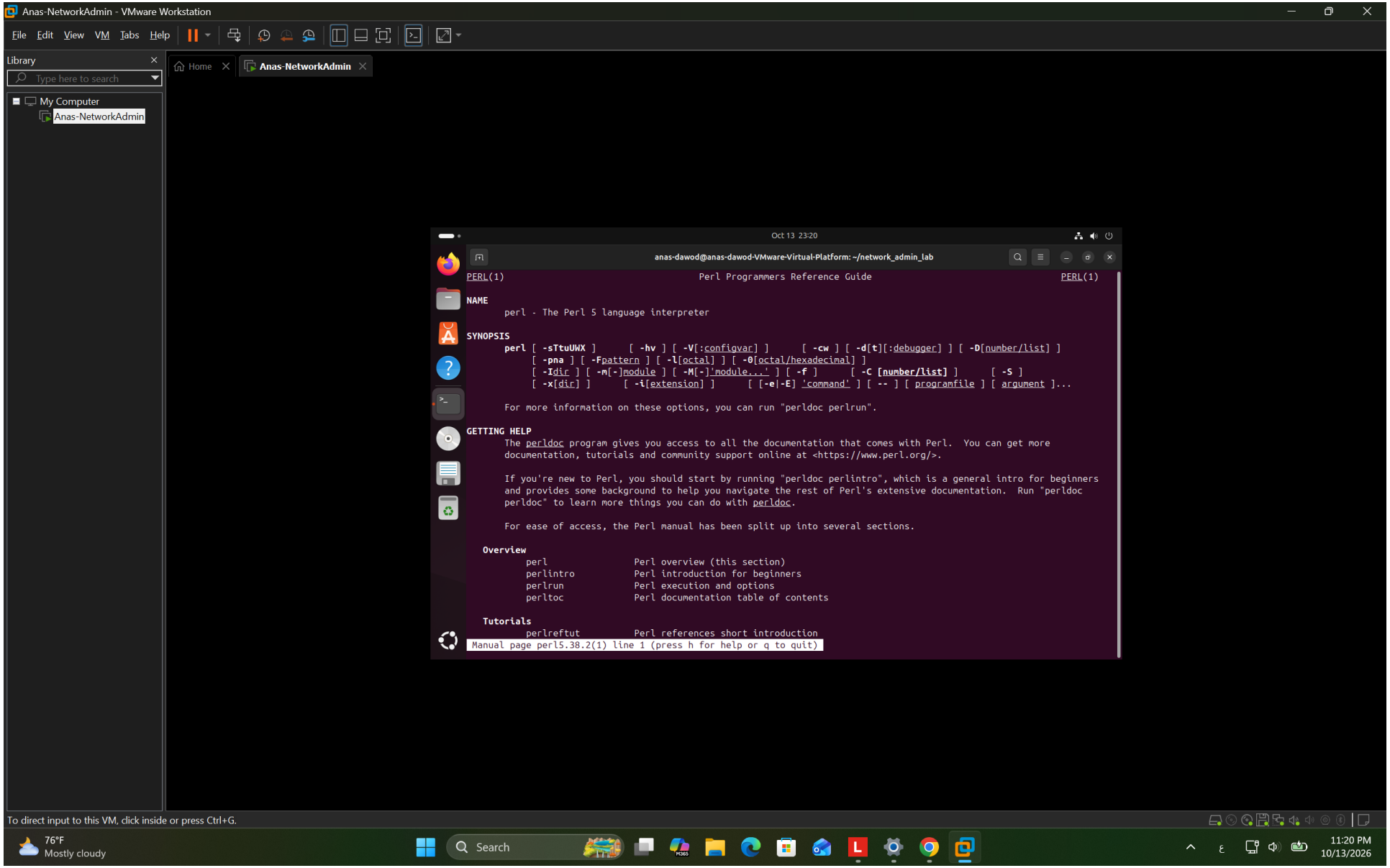
Linux Terminal Evidence 02 / 37

Screenshot 2026-10-13 231640



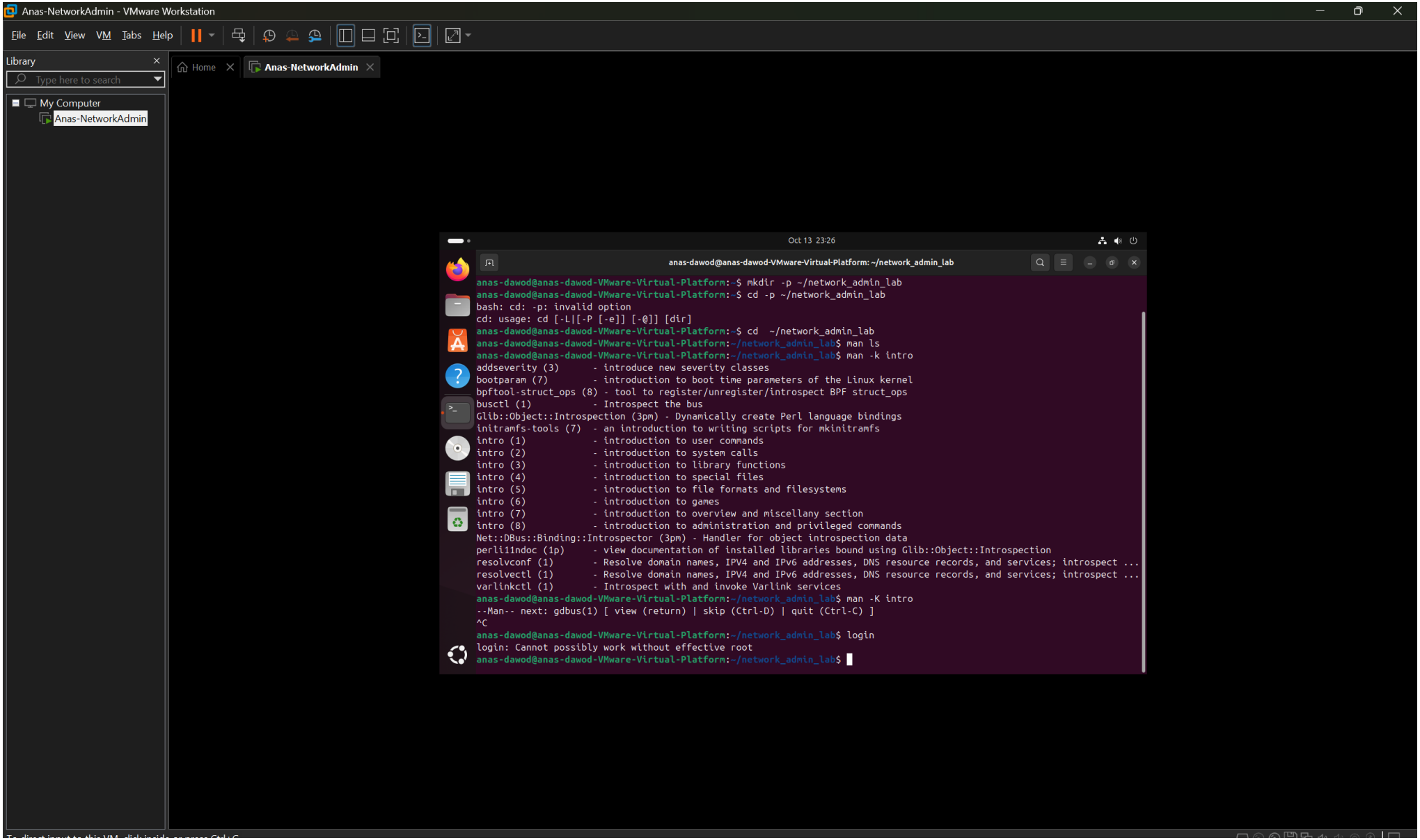
Linux Terminal Evidence 03 / 37

Screenshot 2026-10-13 232057



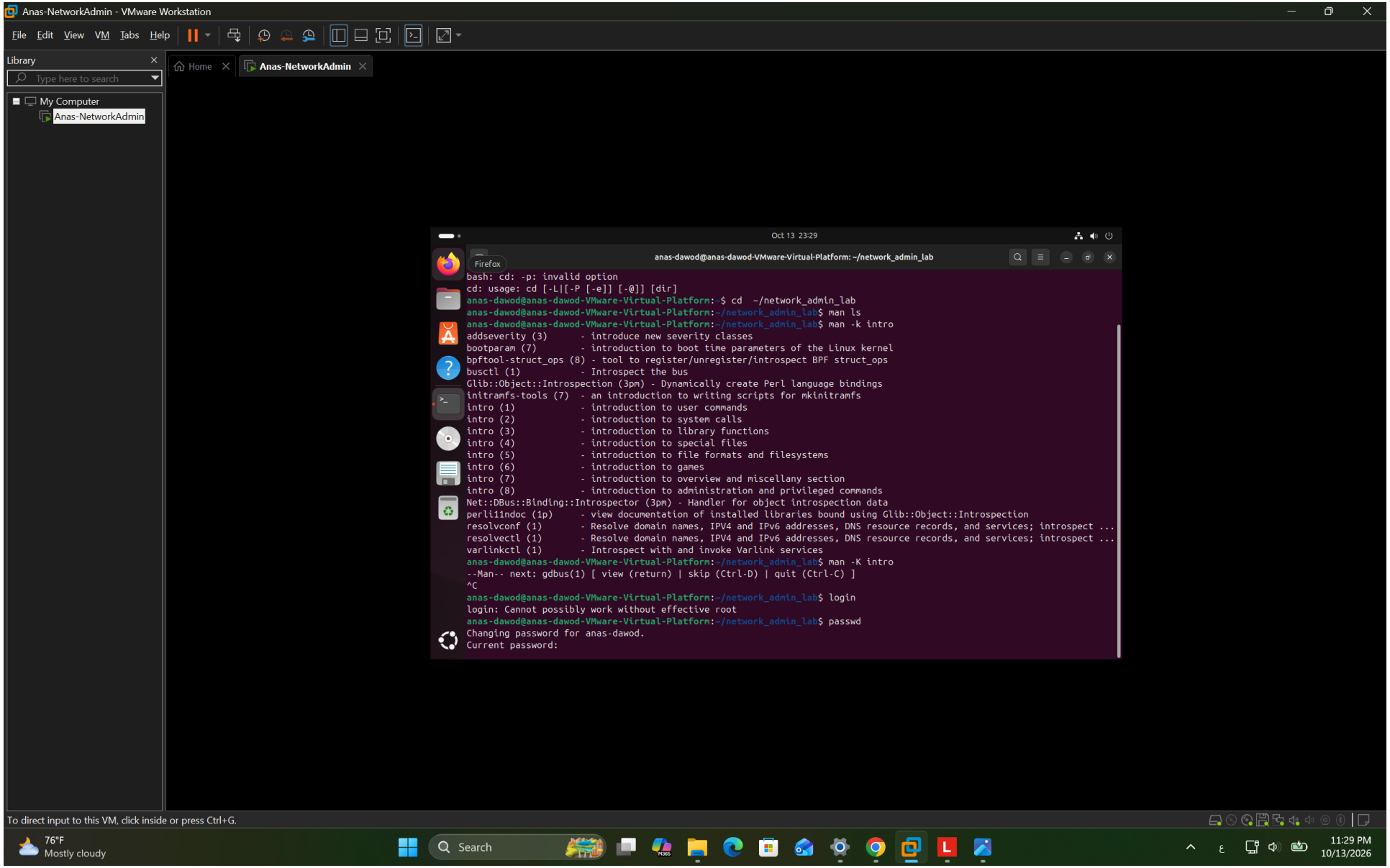
Linux Terminal Evidence 04 / 37

Screenshot 2026-10-13 232618



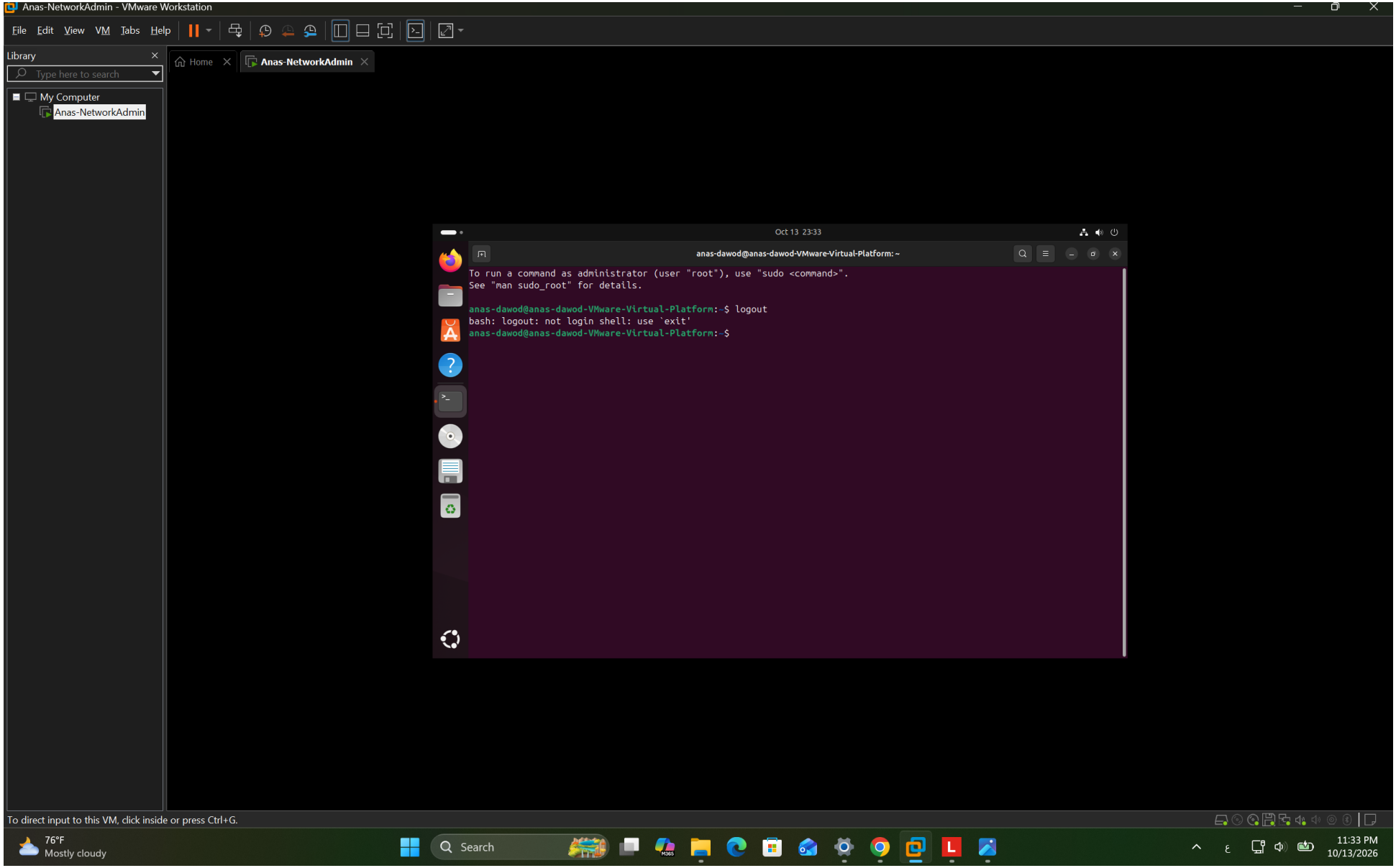
Linux Terminal Evidence 05 / 37

Screenshot 2026-10-13 232933



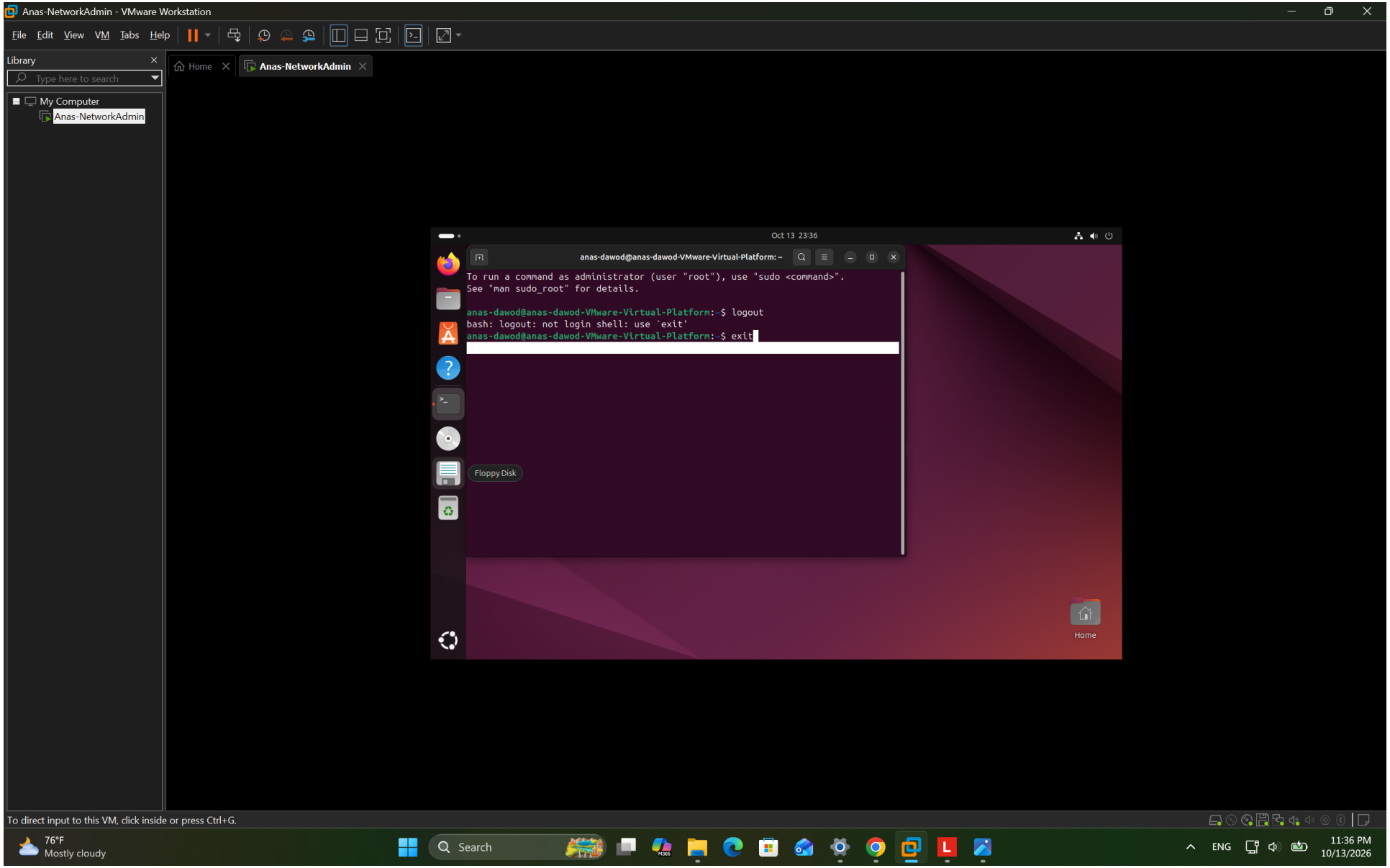
Linux Terminal Evidence 06 / 37

Screenshot 2026-10-13 233343



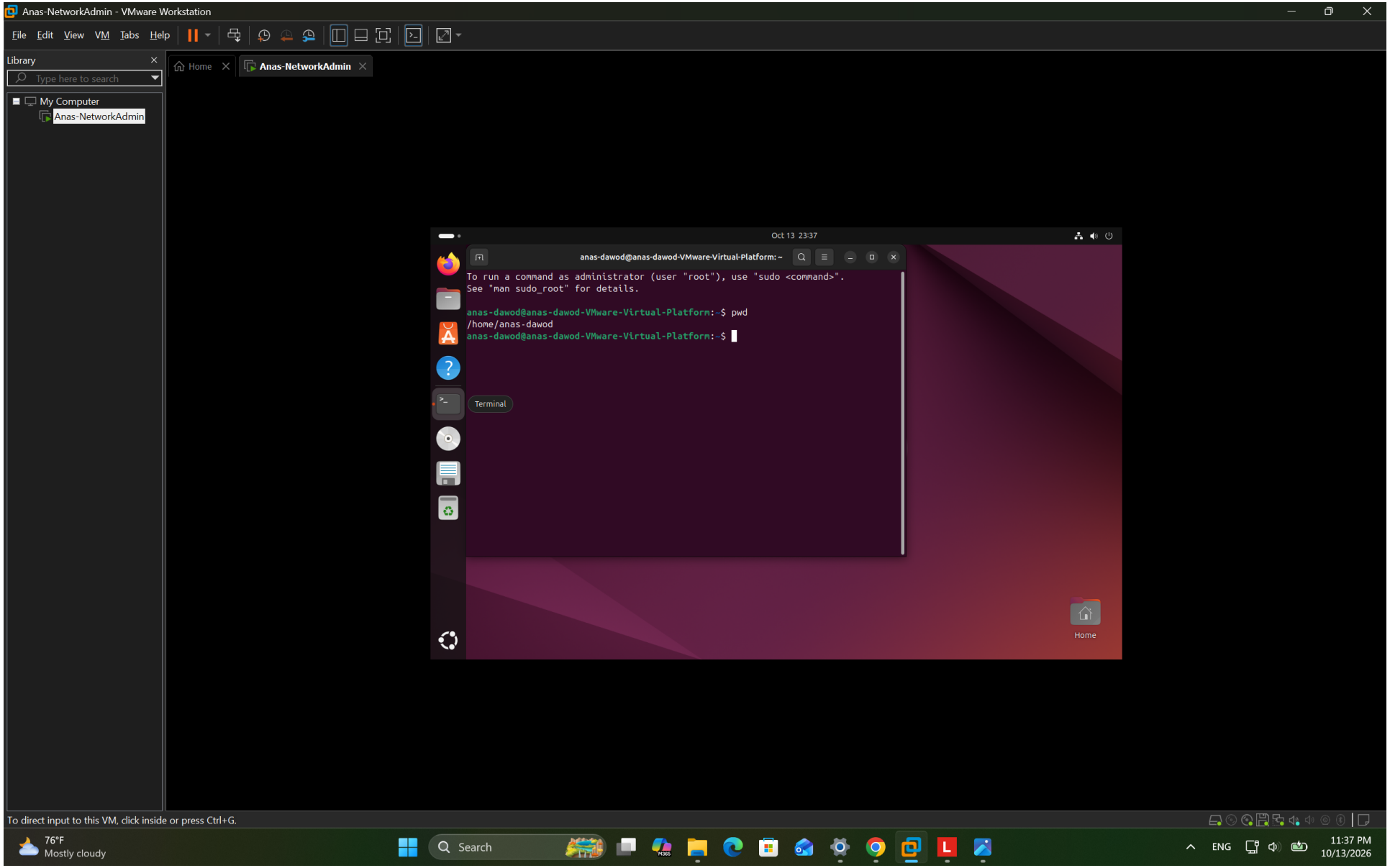
Linux Terminal Evidence 07 / 37

Screenshot 2026-10-13 233609



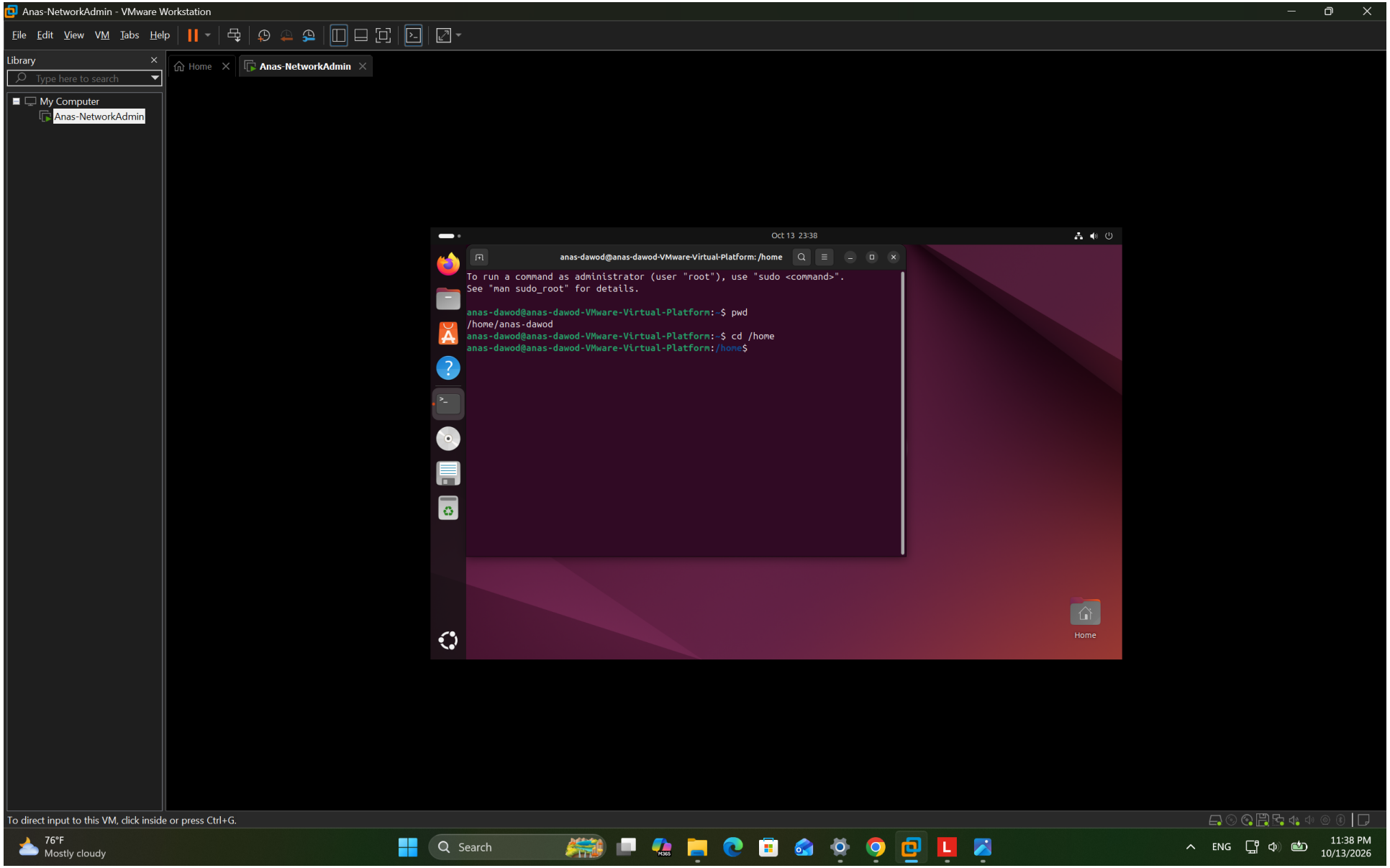
Linux Terminal Evidence 08 / 37

Screenshot 2026-10-13 233726



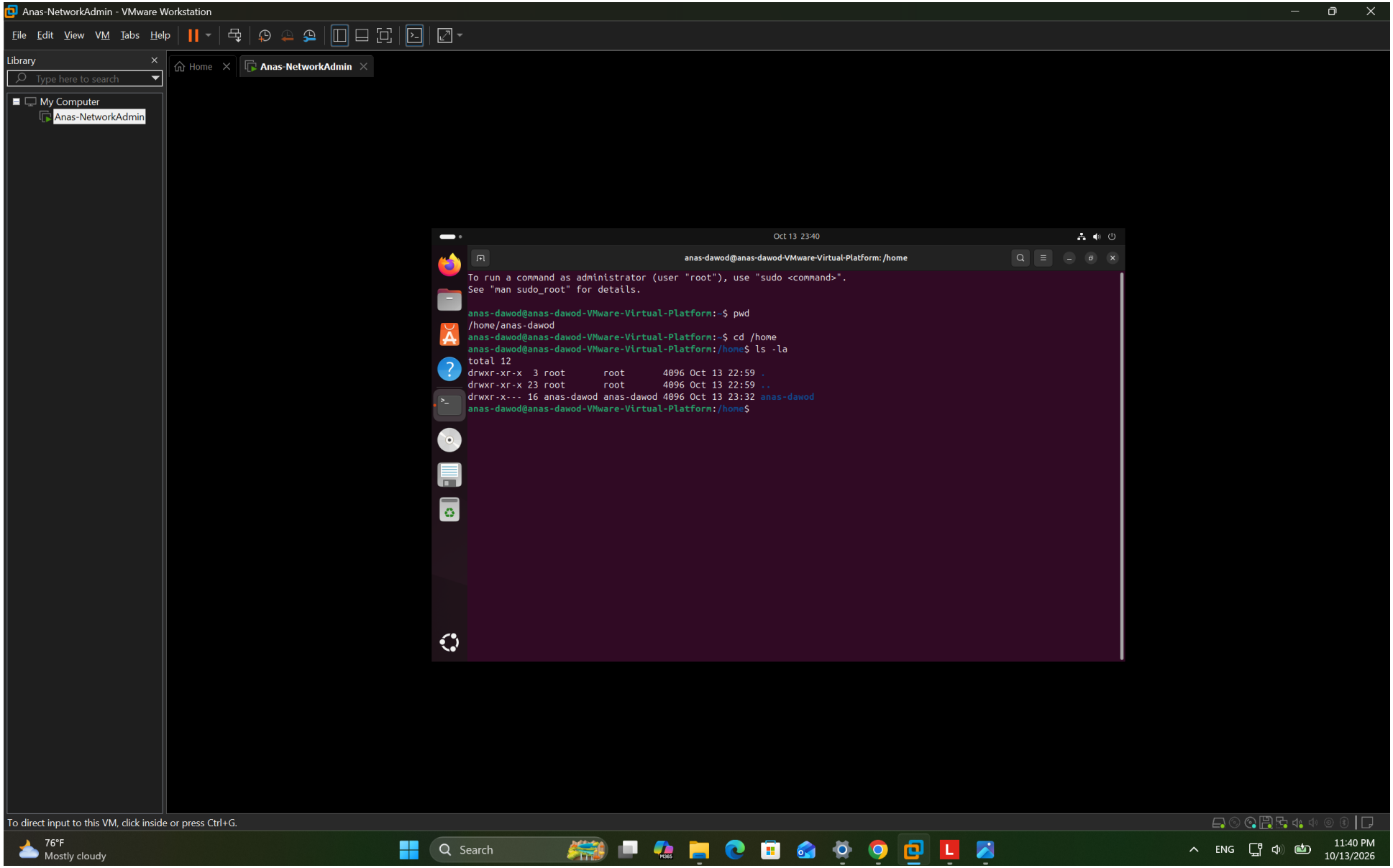
Linux Terminal Evidence 09 / 37

Screenshot 2026-10-13 233900



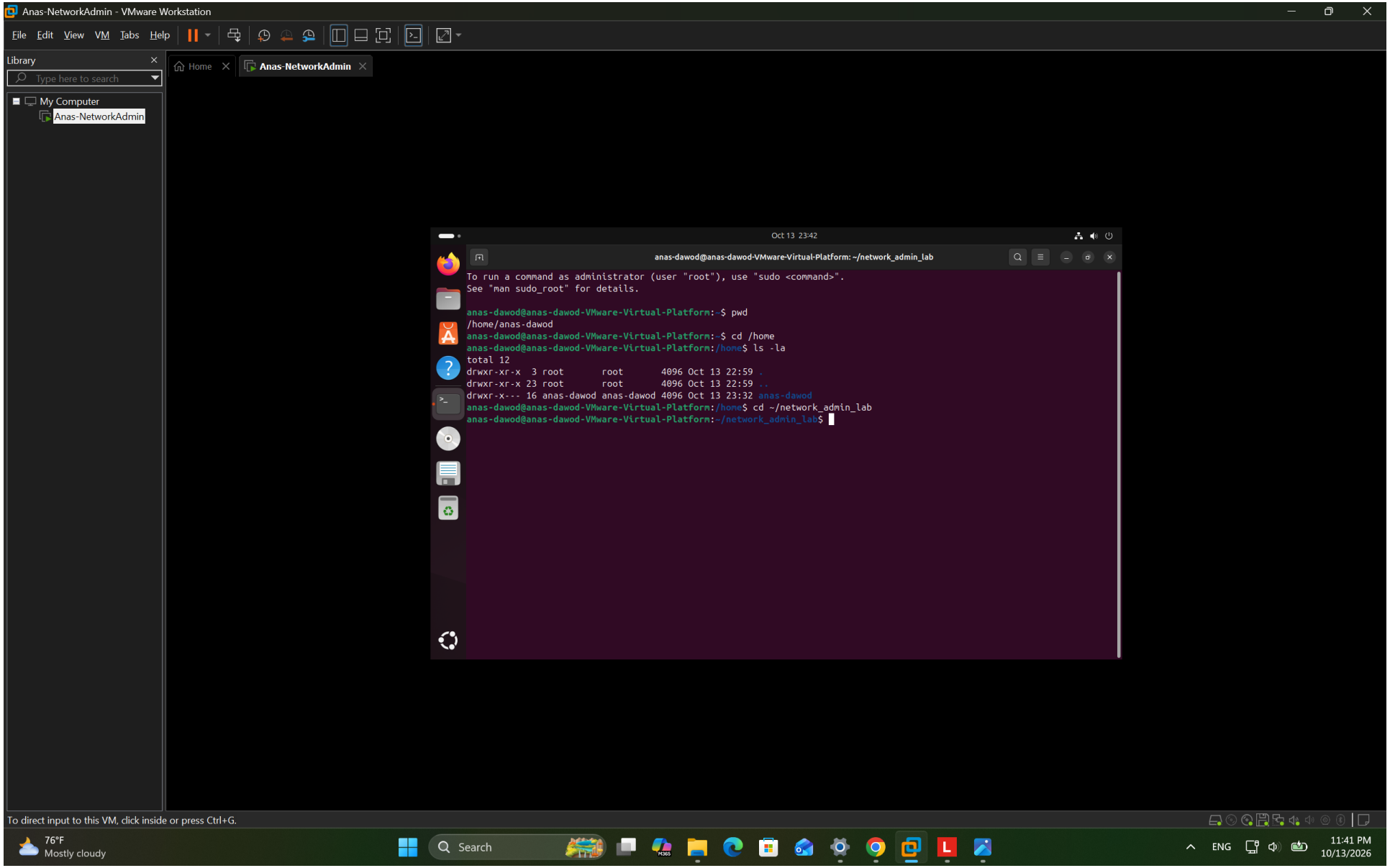
Linux Terminal Evidence 10 / 37

Screenshot 2026-10-13 234014



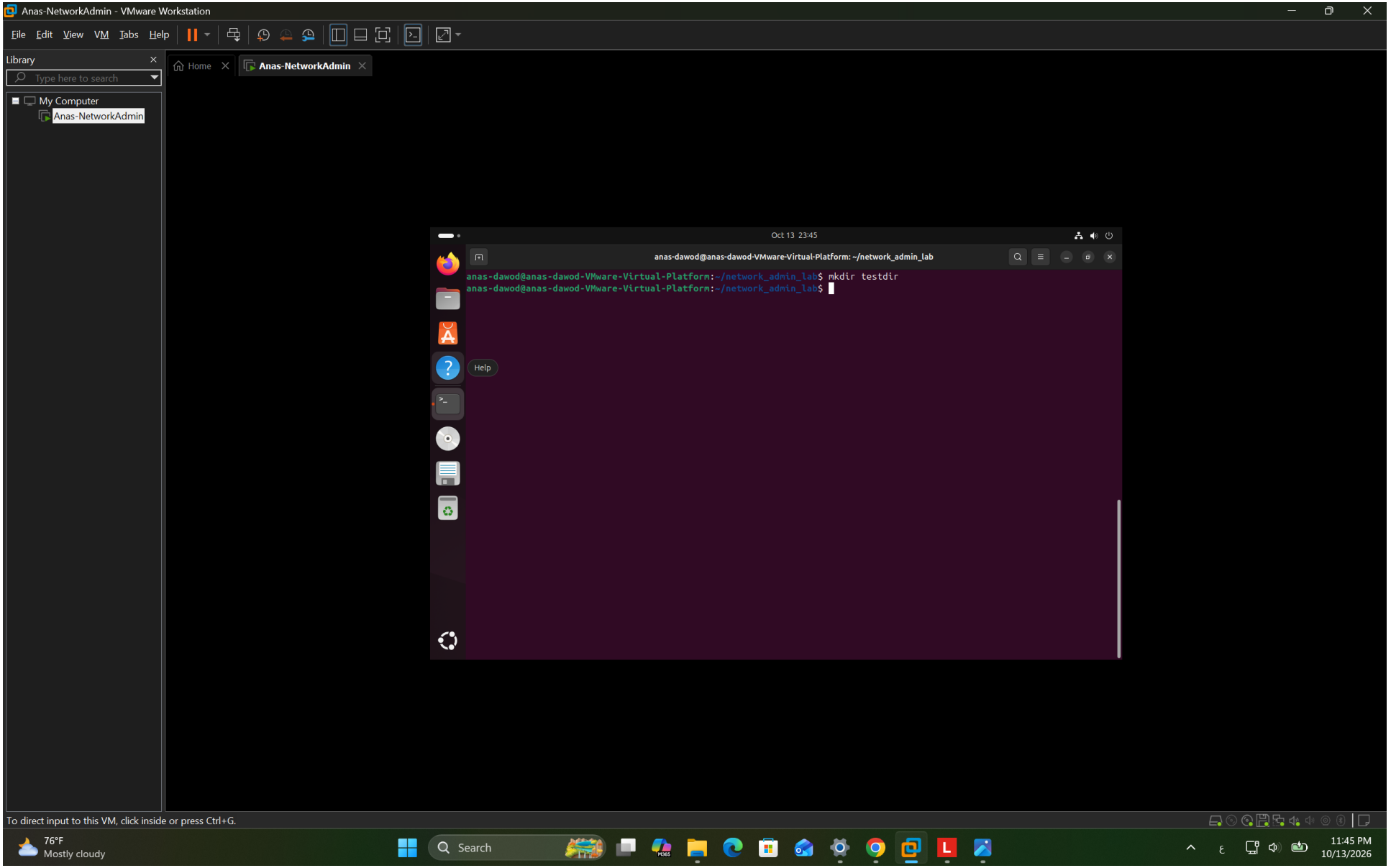
Linux Terminal Evidence 11 / 37

Screenshot 2026-10-13 234205



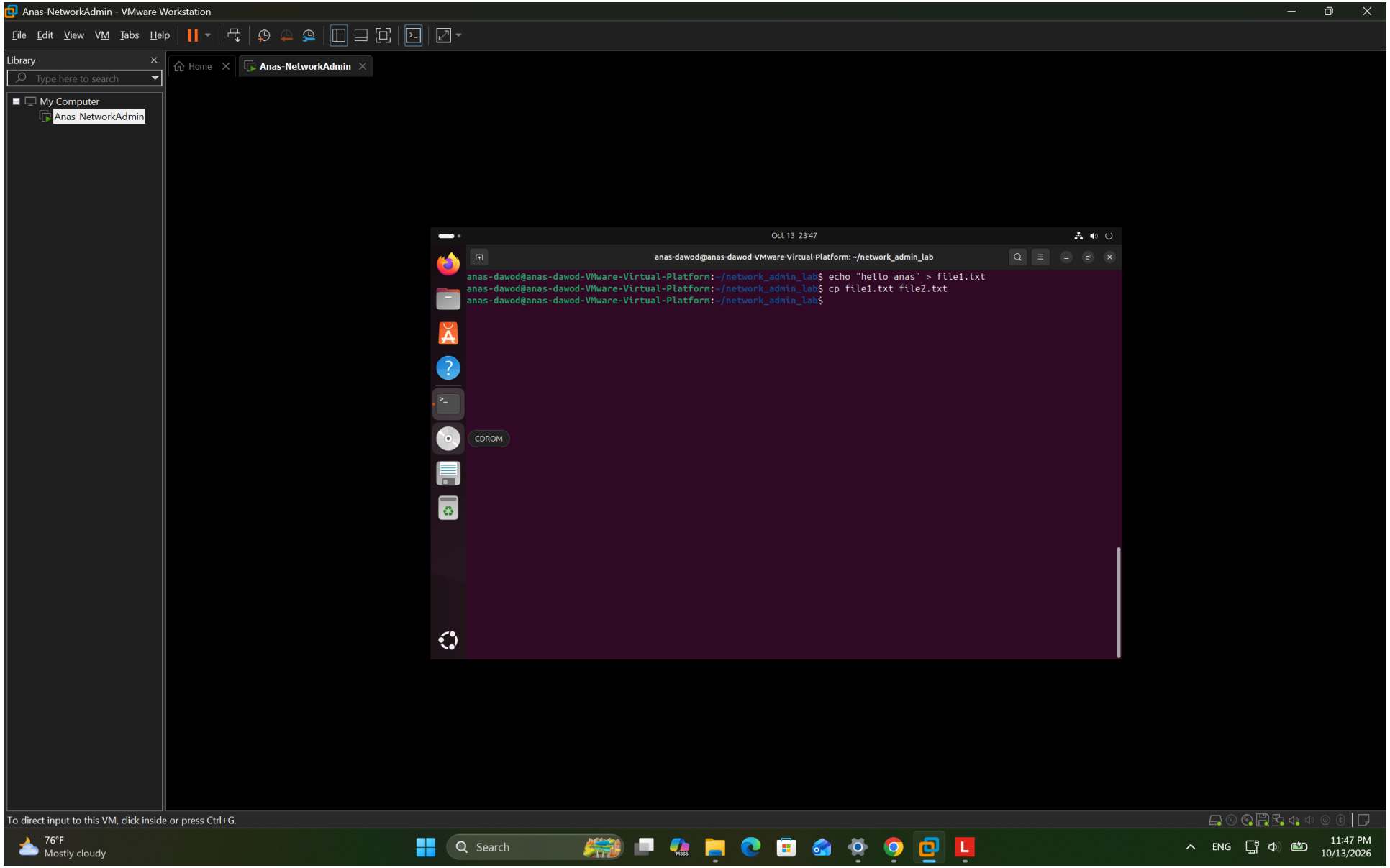
Linux Terminal Evidence 12 / 37

Screenshot 2026-10-13 234535



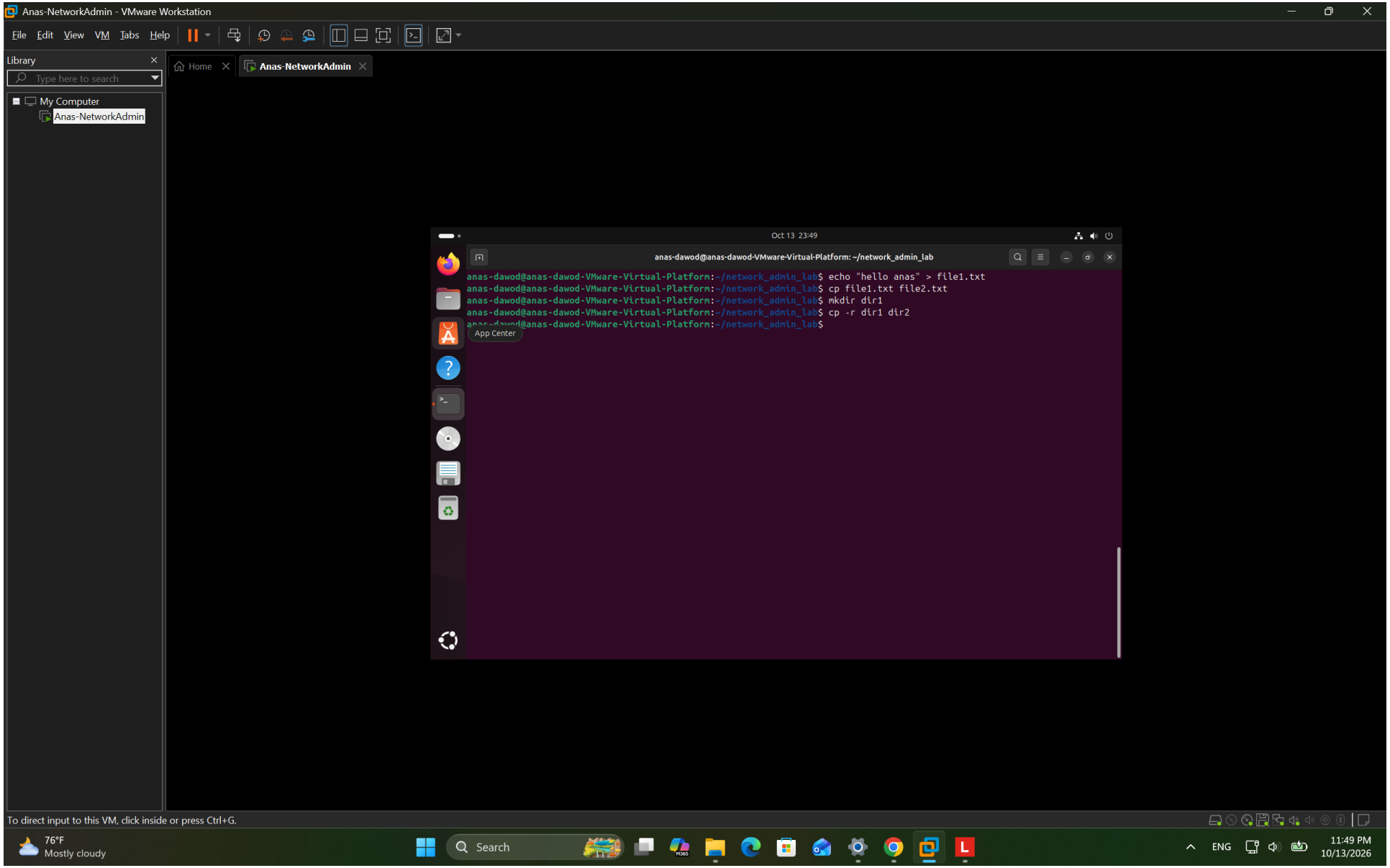
Linux Terminal Evidence 13 / 37

Screenshot 2026-10-13 234733



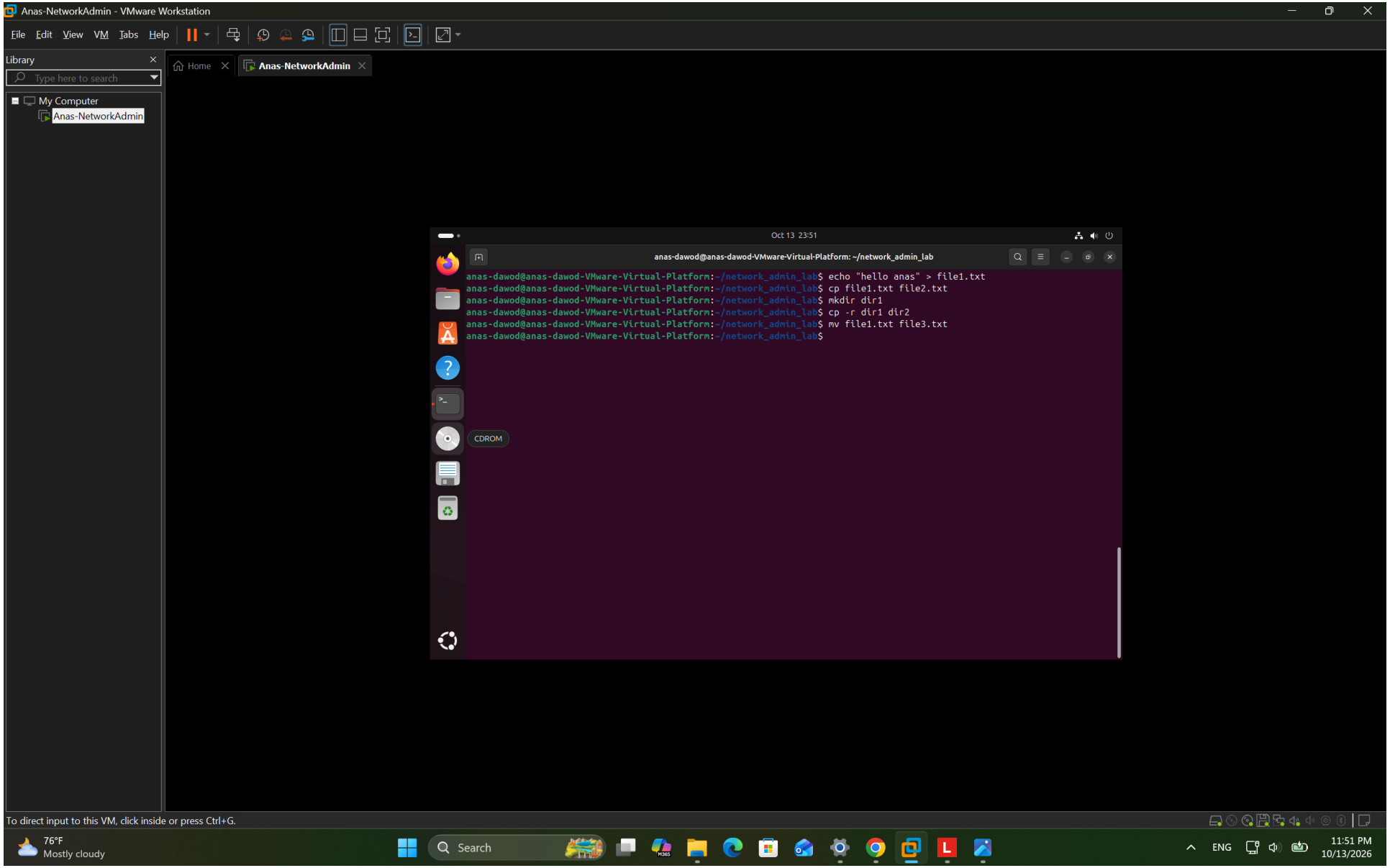
Linux Terminal Evidence 14 / 37

Screenshot 2026-10-13 234956



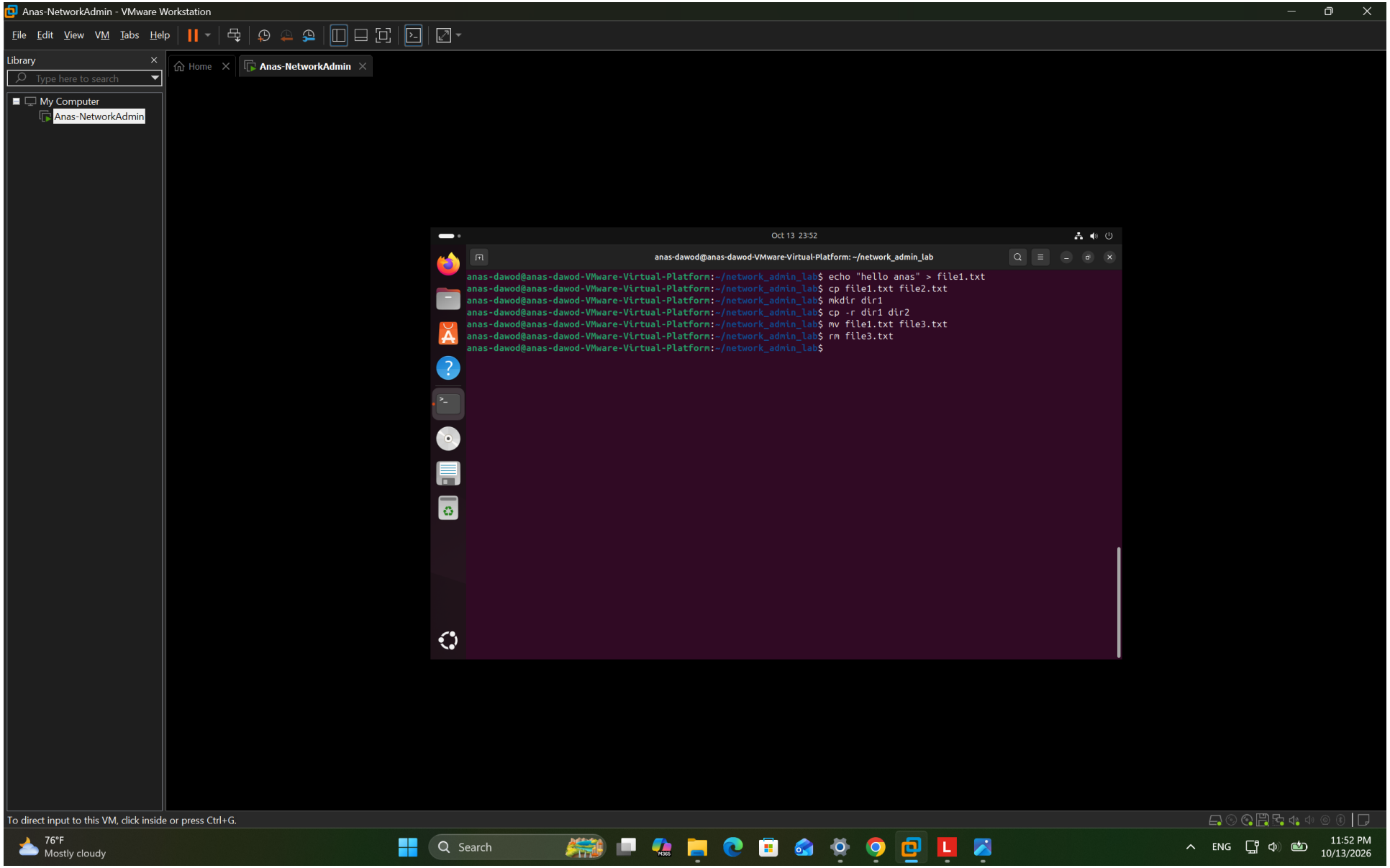
Linux Terminal Evidence 15 / 37

Screenshot 2026-10-13 235131



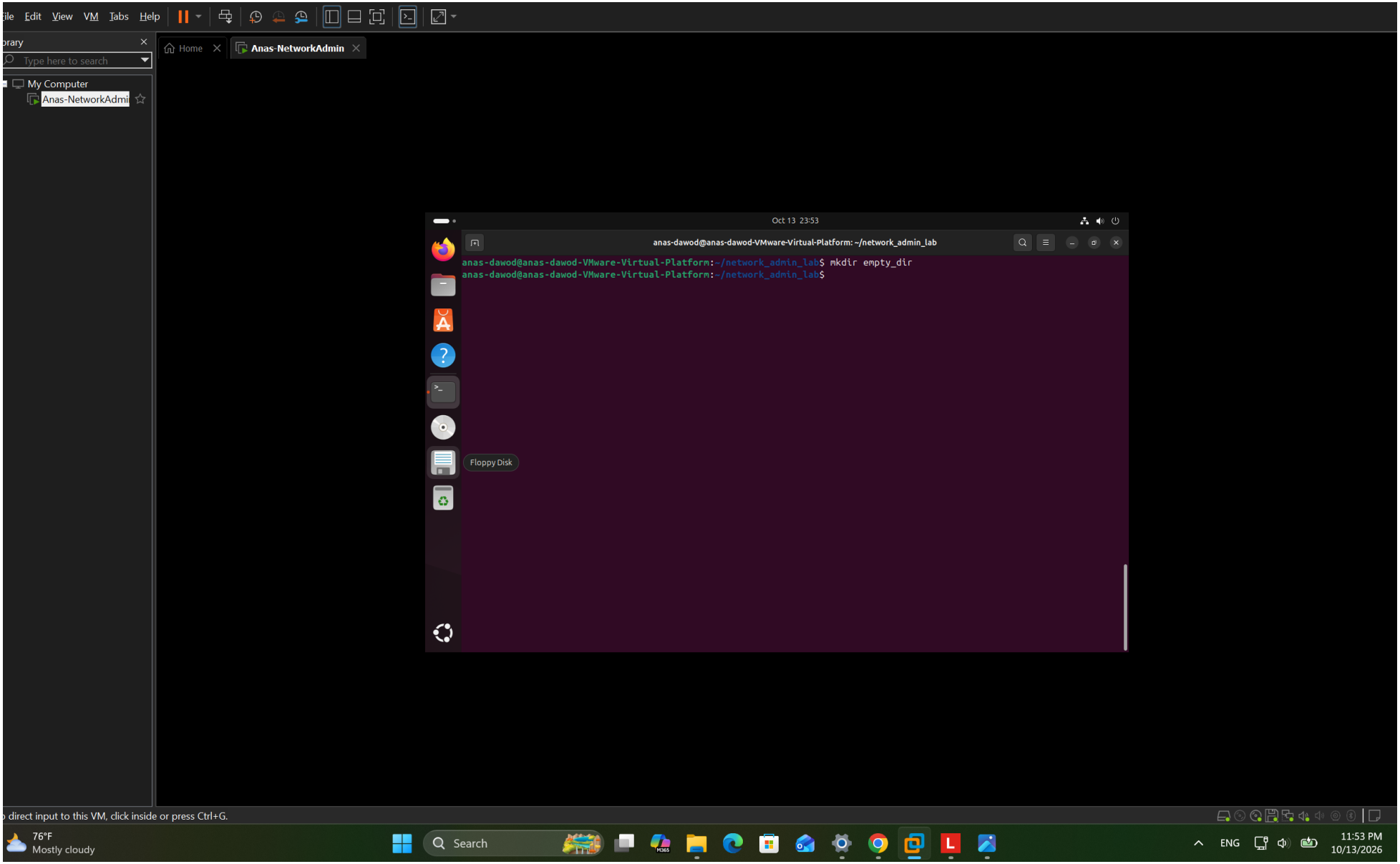
Linux Terminal Evidence 16 / 37

Screenshot 2026-10-13 235228



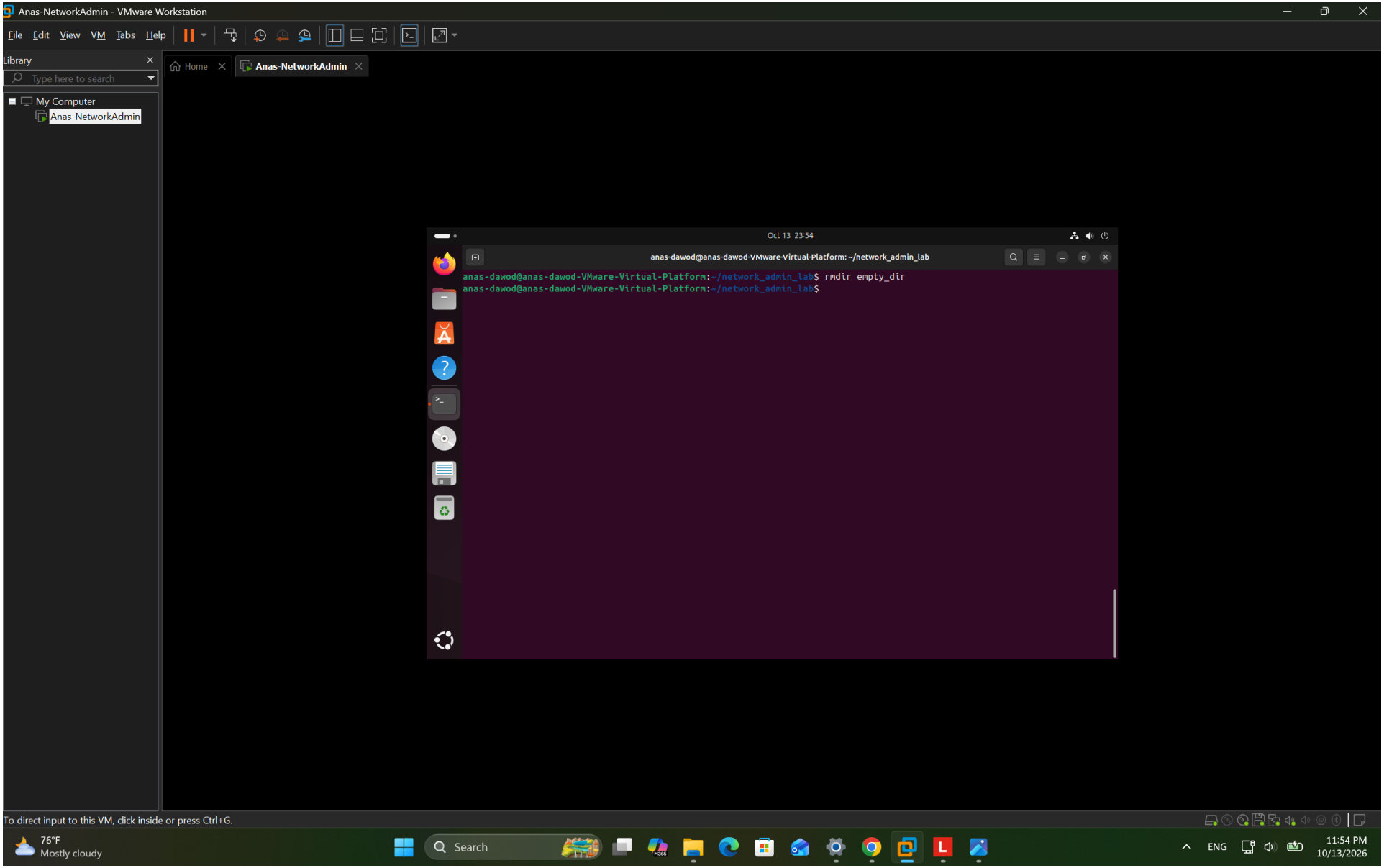
Linux Terminal Evidence 17 / 37

Screenshot 2026-10-13 235334



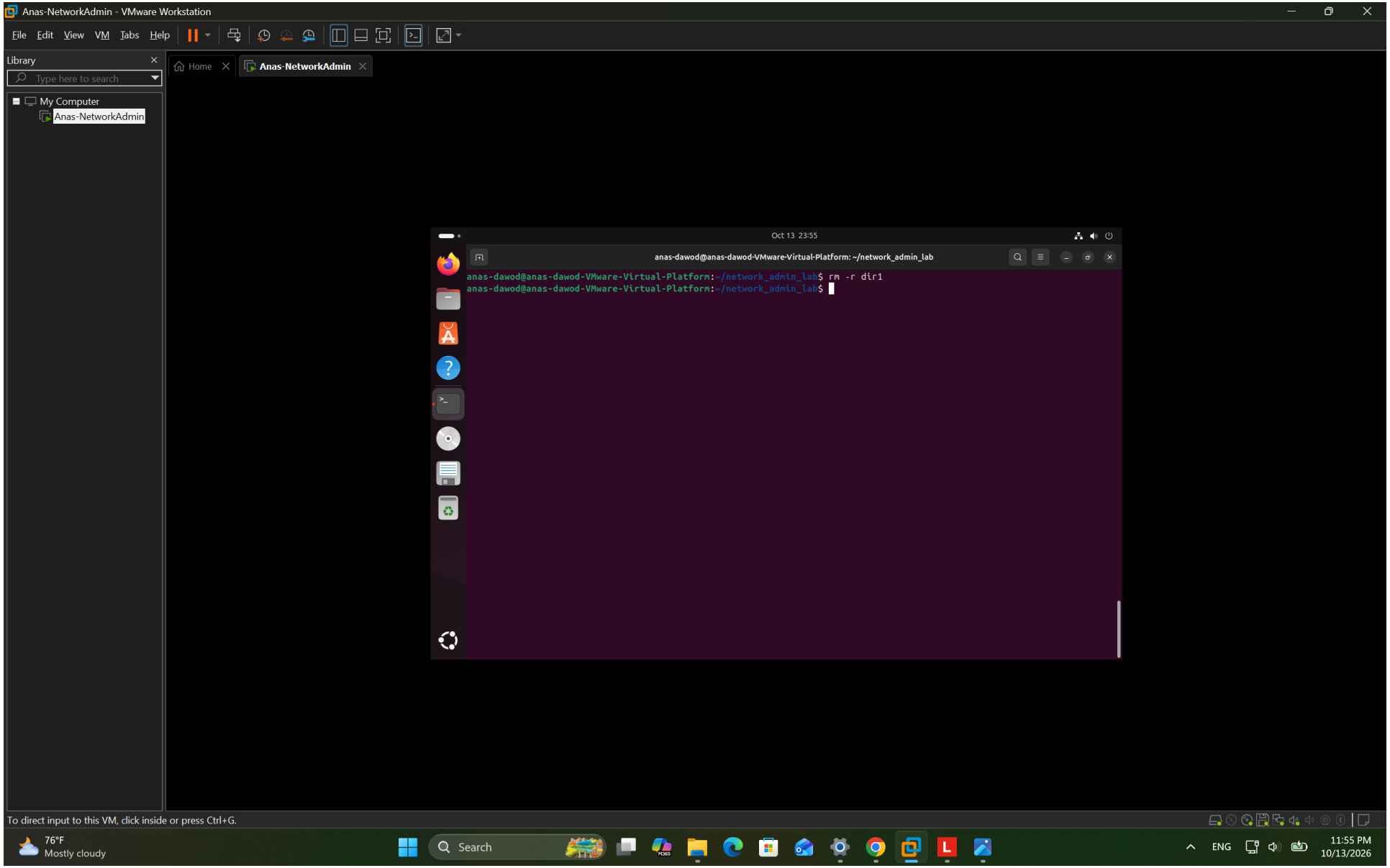
Linux Terminal Evidence 18 / 37

Screenshot 2026-10-13 235440



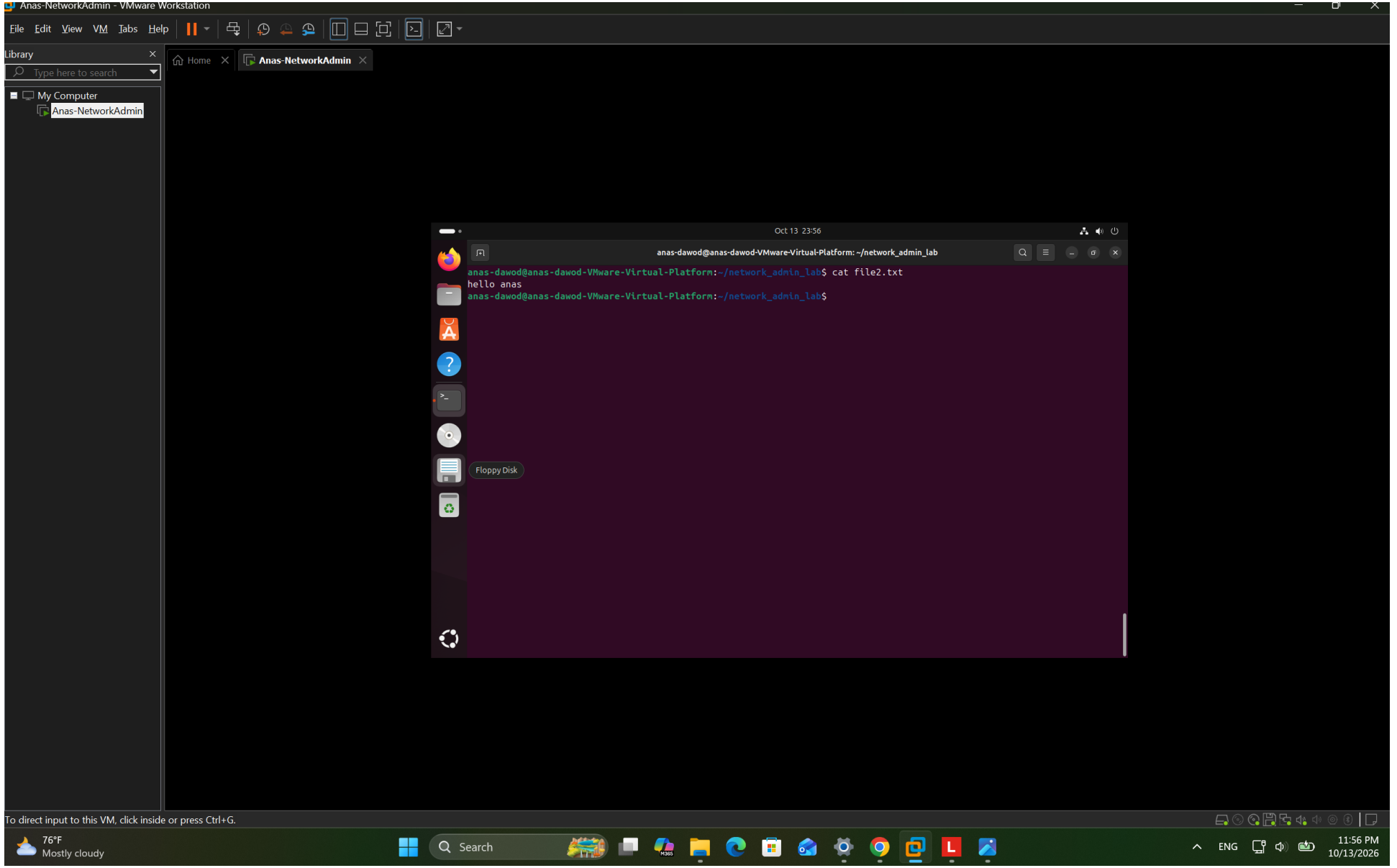
Linux Terminal Evidence 19 / 37

Screenshot 2026-10-13 235521



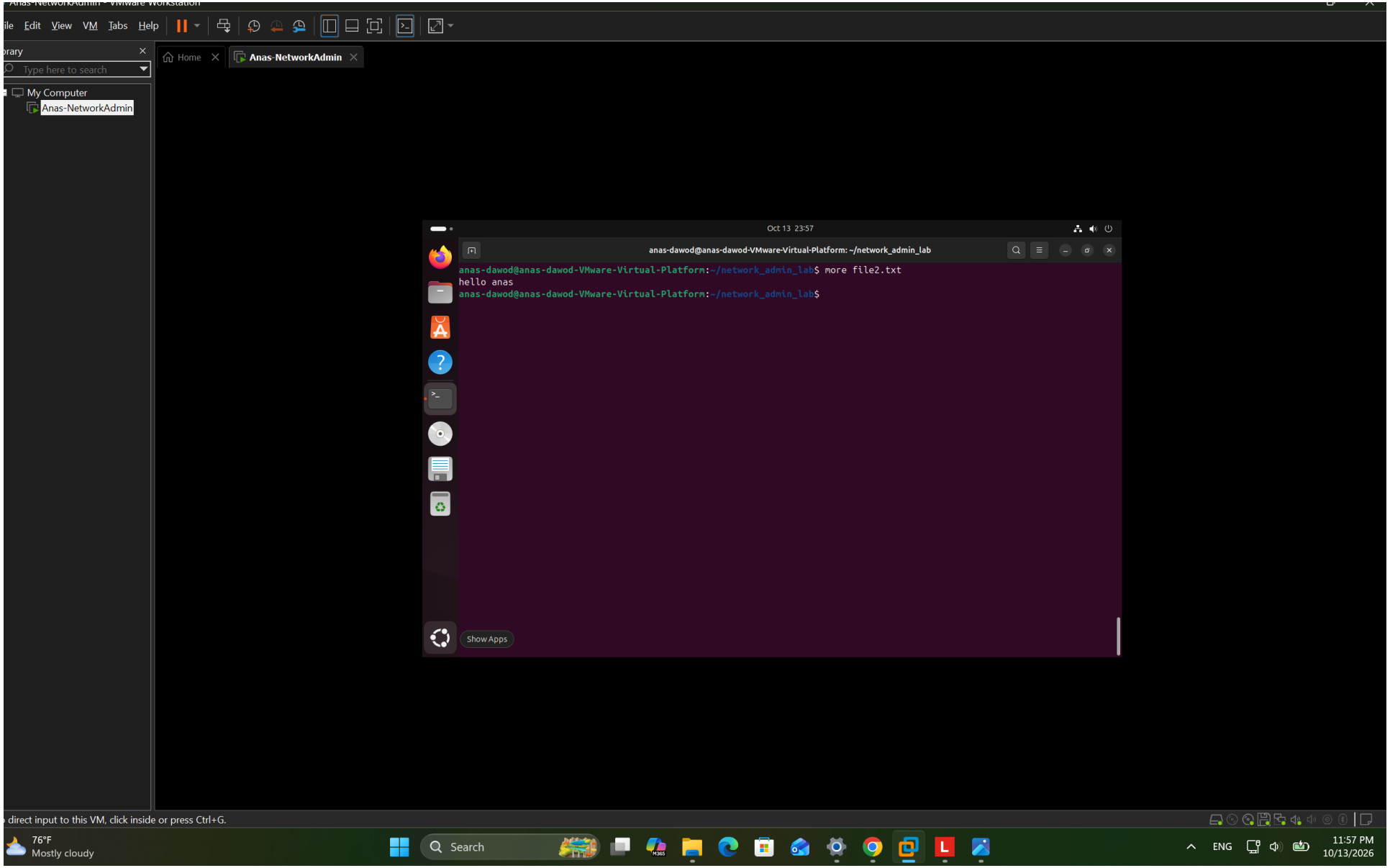
Linux Terminal Evidence 20 / 37

Screenshot 2026-10-13 235623



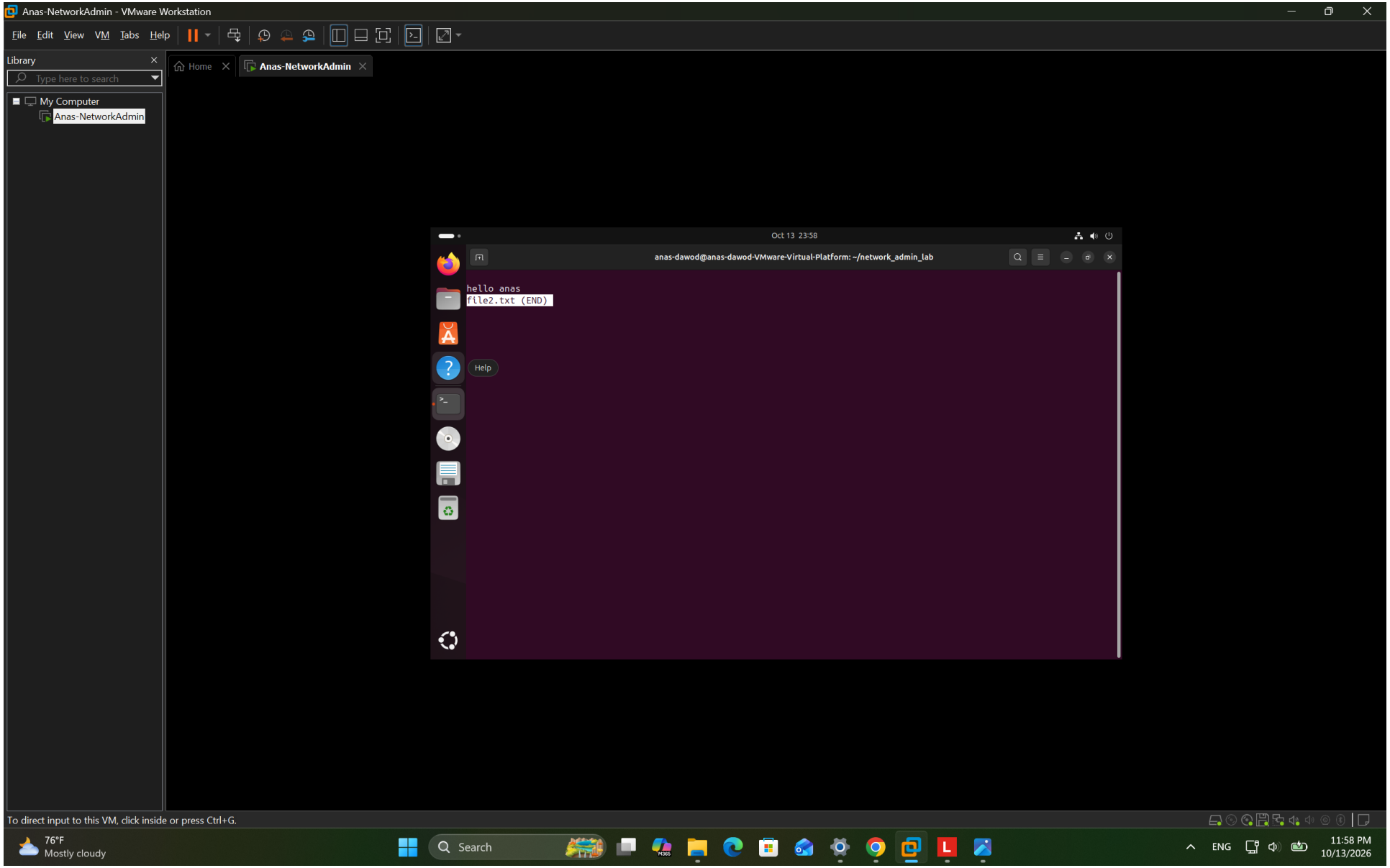
Linux Terminal Evidence 21 / 37

Screenshot 2026-10-13 235704



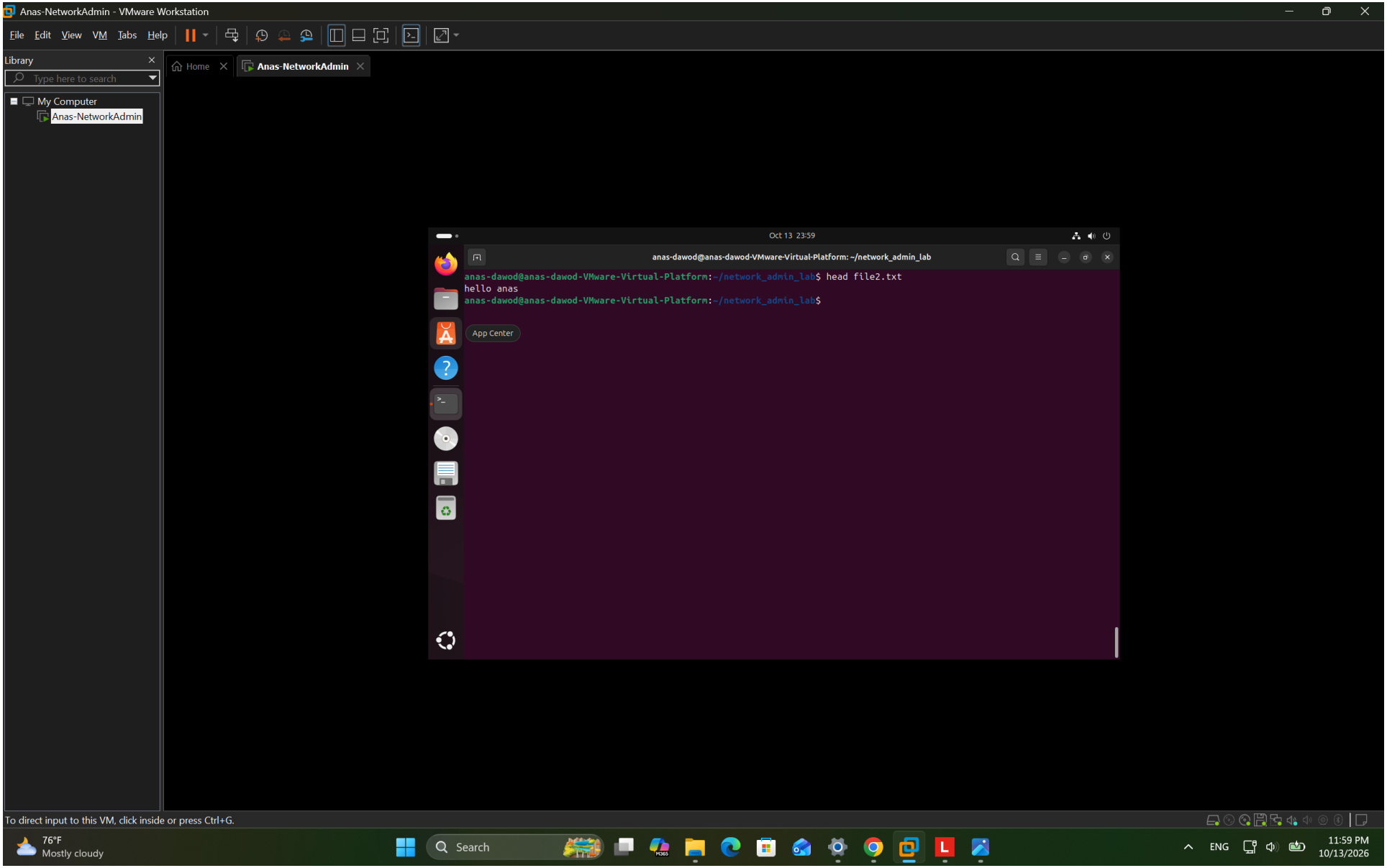
Linux Terminal Evidence 22 / 37

Screenshot 2026-10-13 235818



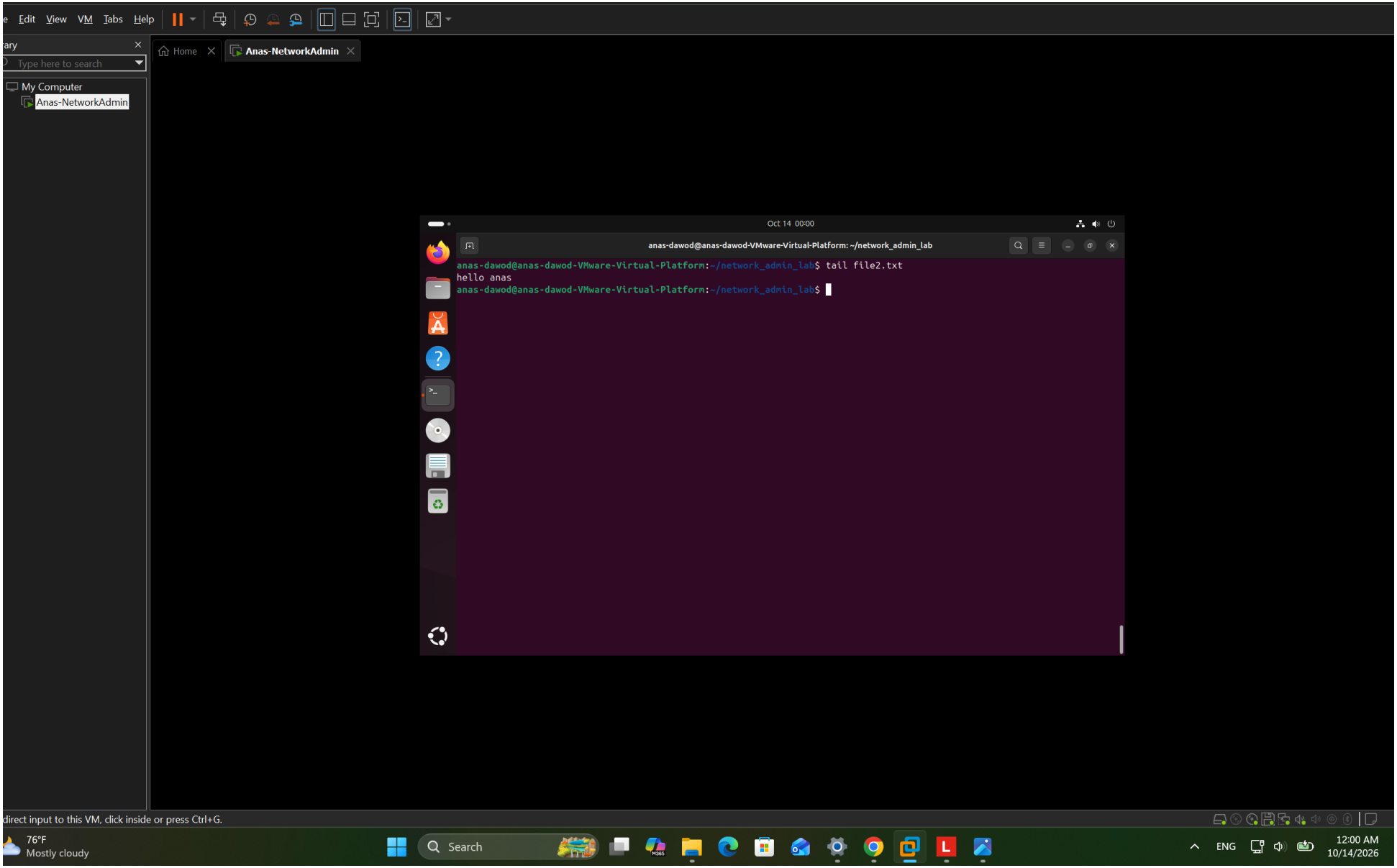
Linux Terminal Evidence 23 / 37

Screenshot 2026-10-13 235925



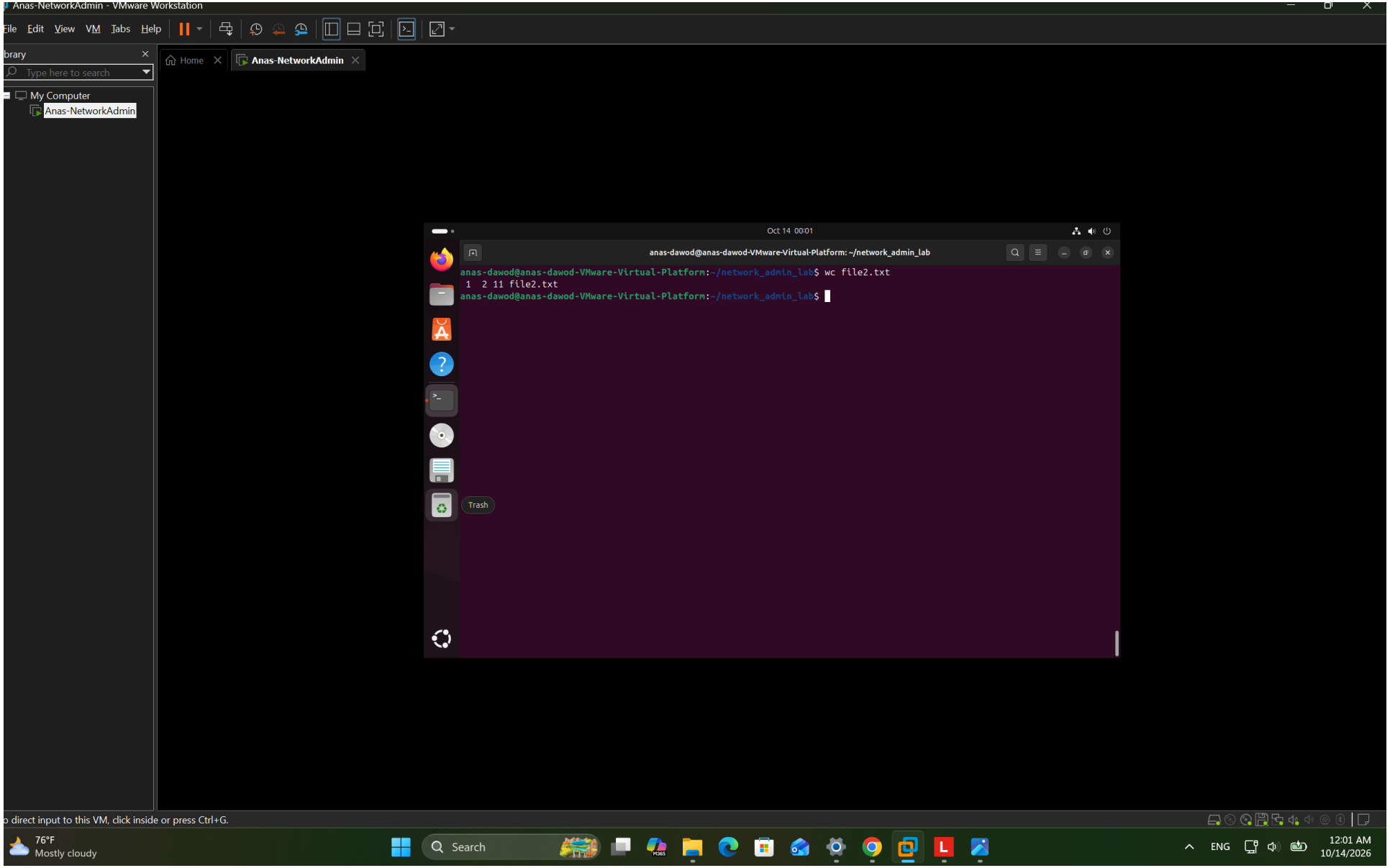
Linux Terminal Evidence 24 / 37

Screenshot 2026-10-14 000022



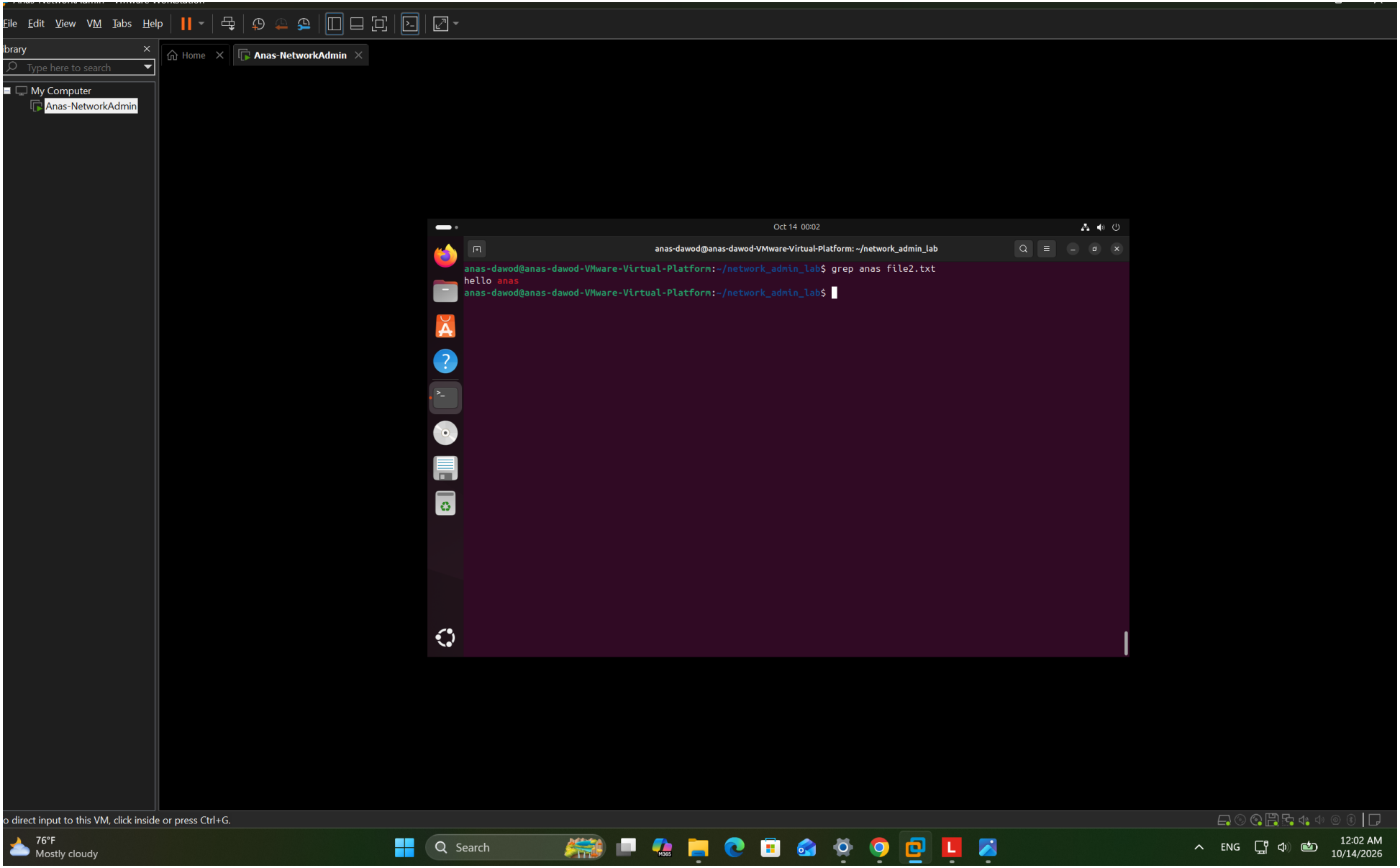
Linux Terminal Evidence 25 / 37

Screenshot 2026-10-14 000103



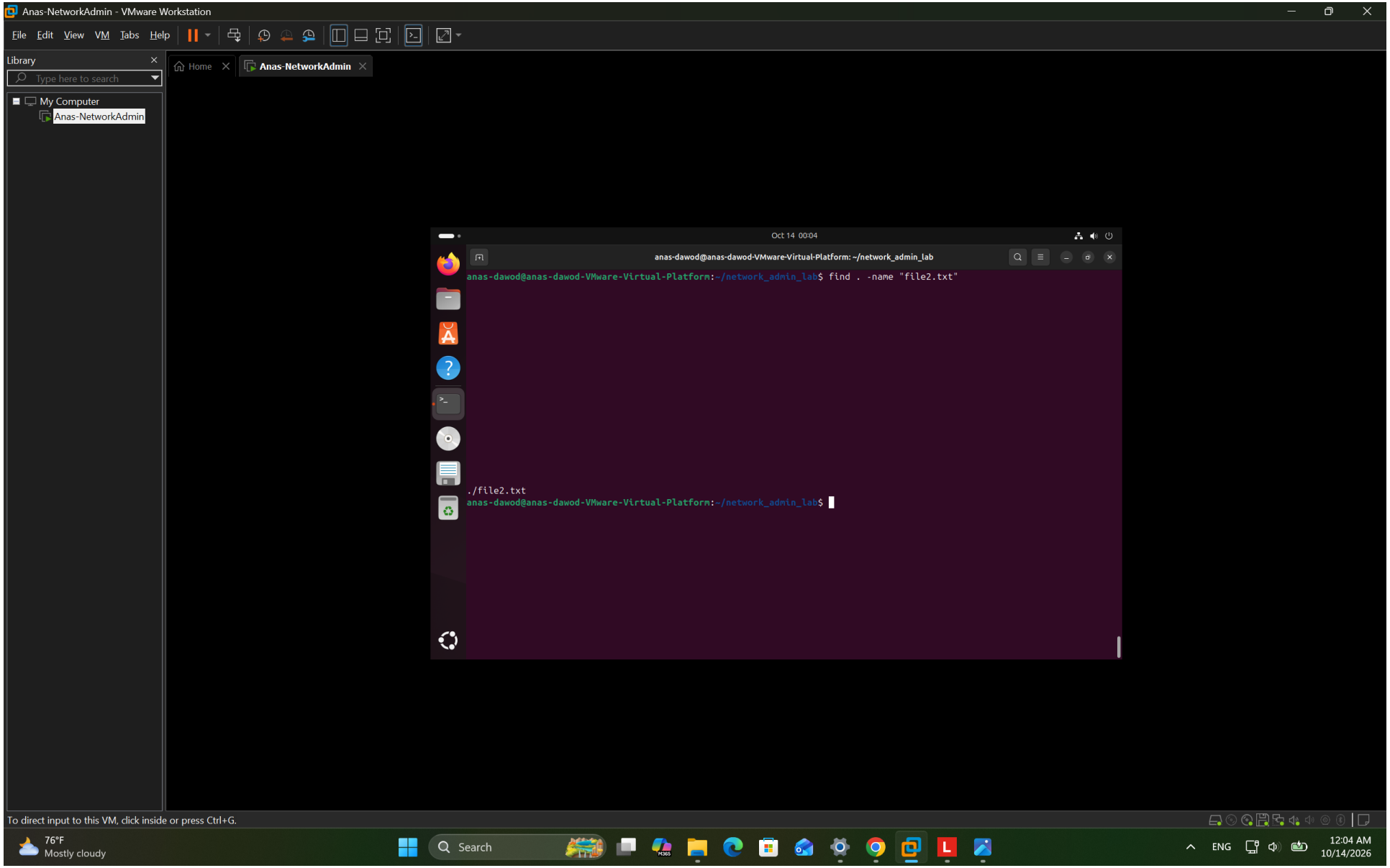
Linux Terminal Evidence 26 / 37

Screenshot 2026-10-14 000228



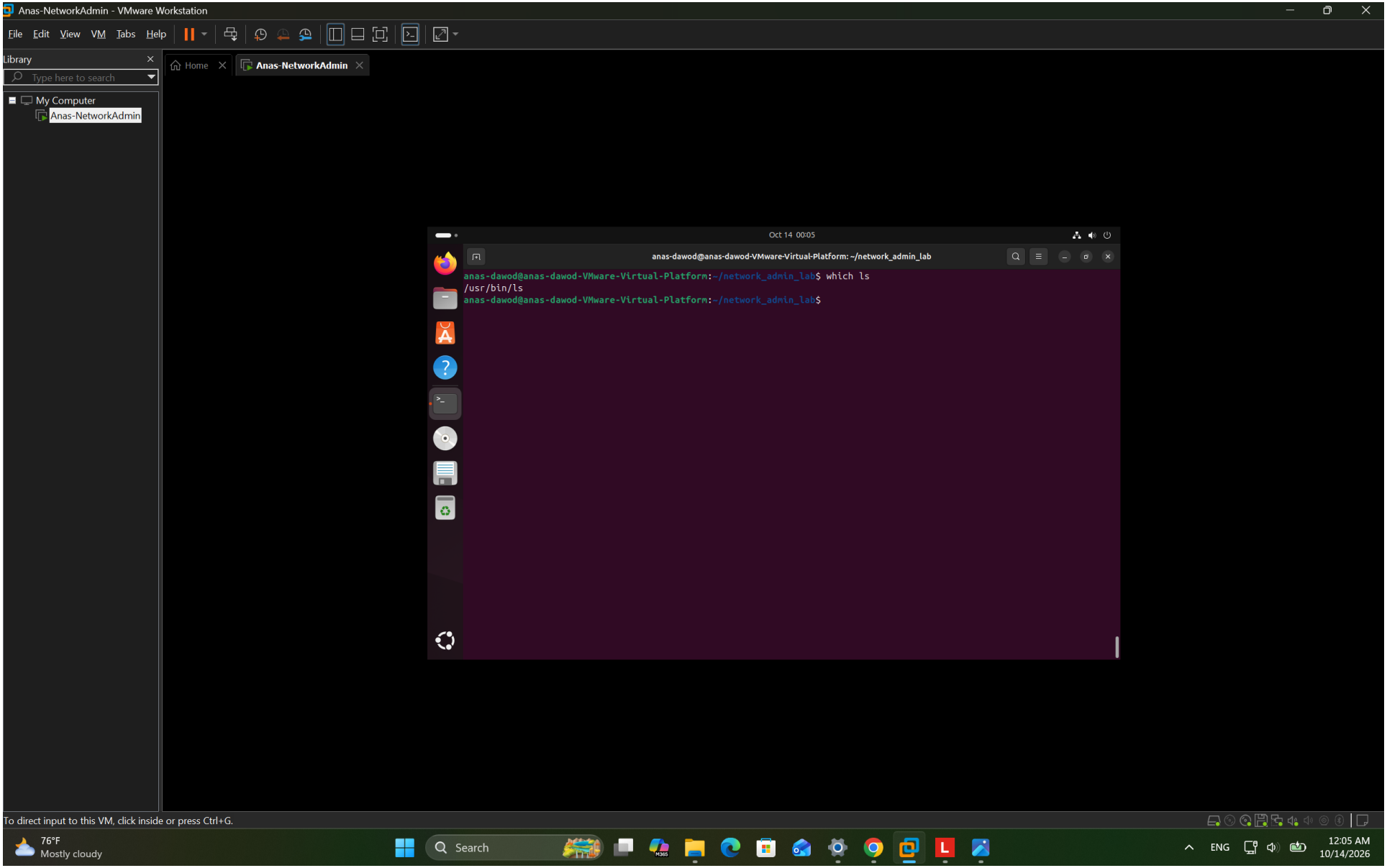
Linux Terminal Evidence 27 / 37

Screenshot 2026-10-14 000417



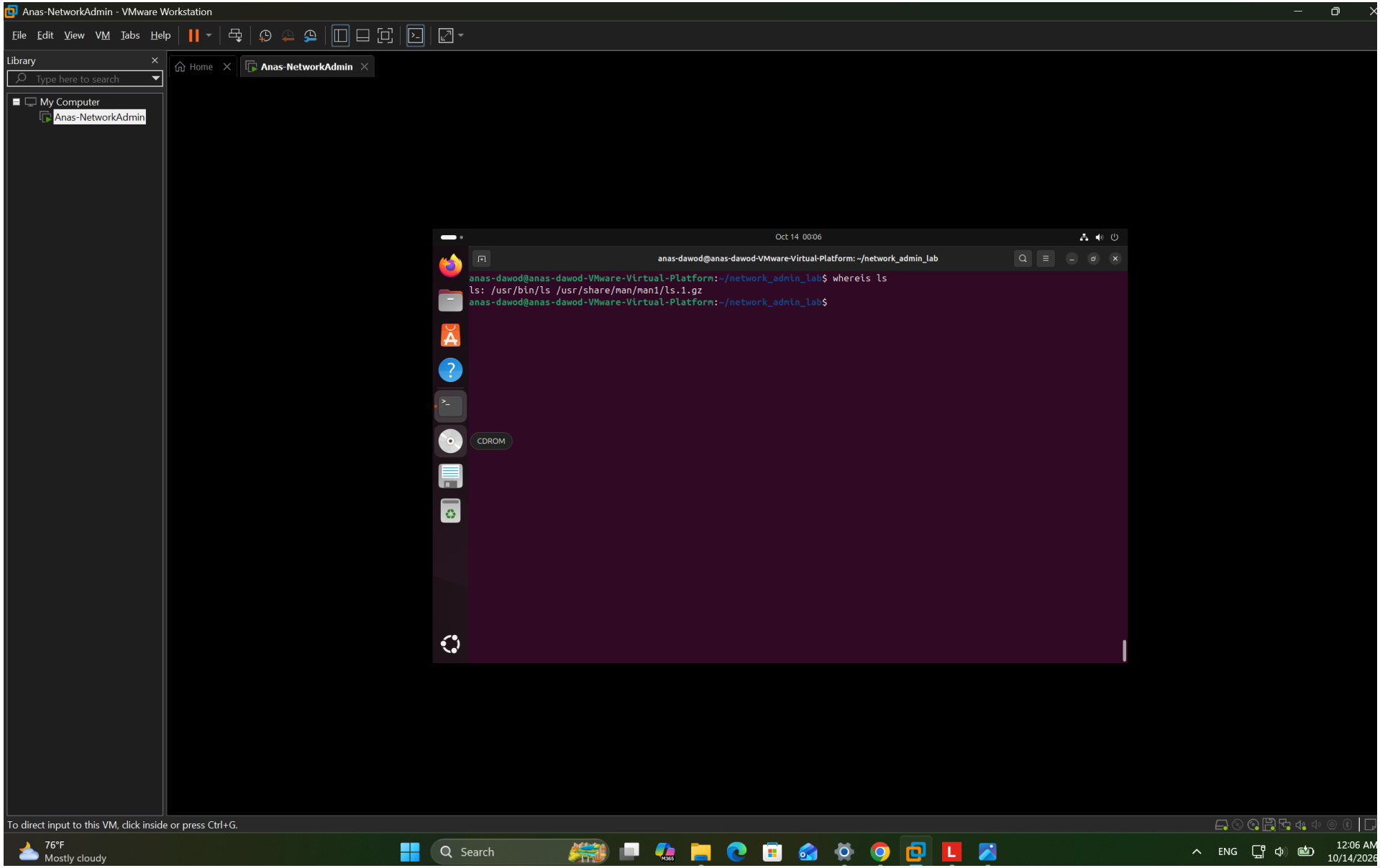
Linux Terminal Evidence 28 / 37

Screenshot 2026-10-14 000504



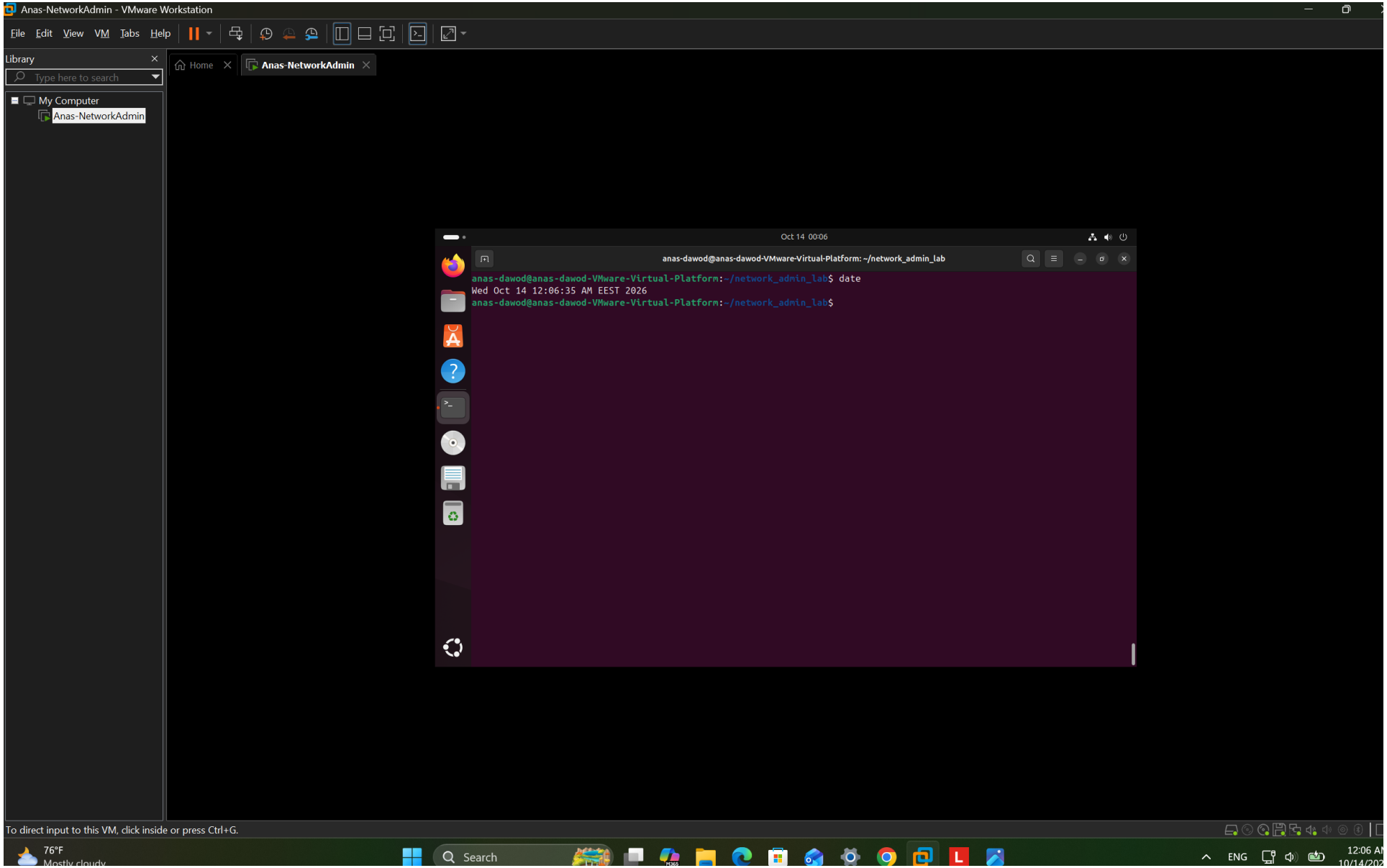
Linux Terminal Evidence 29 / 37

Screenshot 2026-10-14 000607



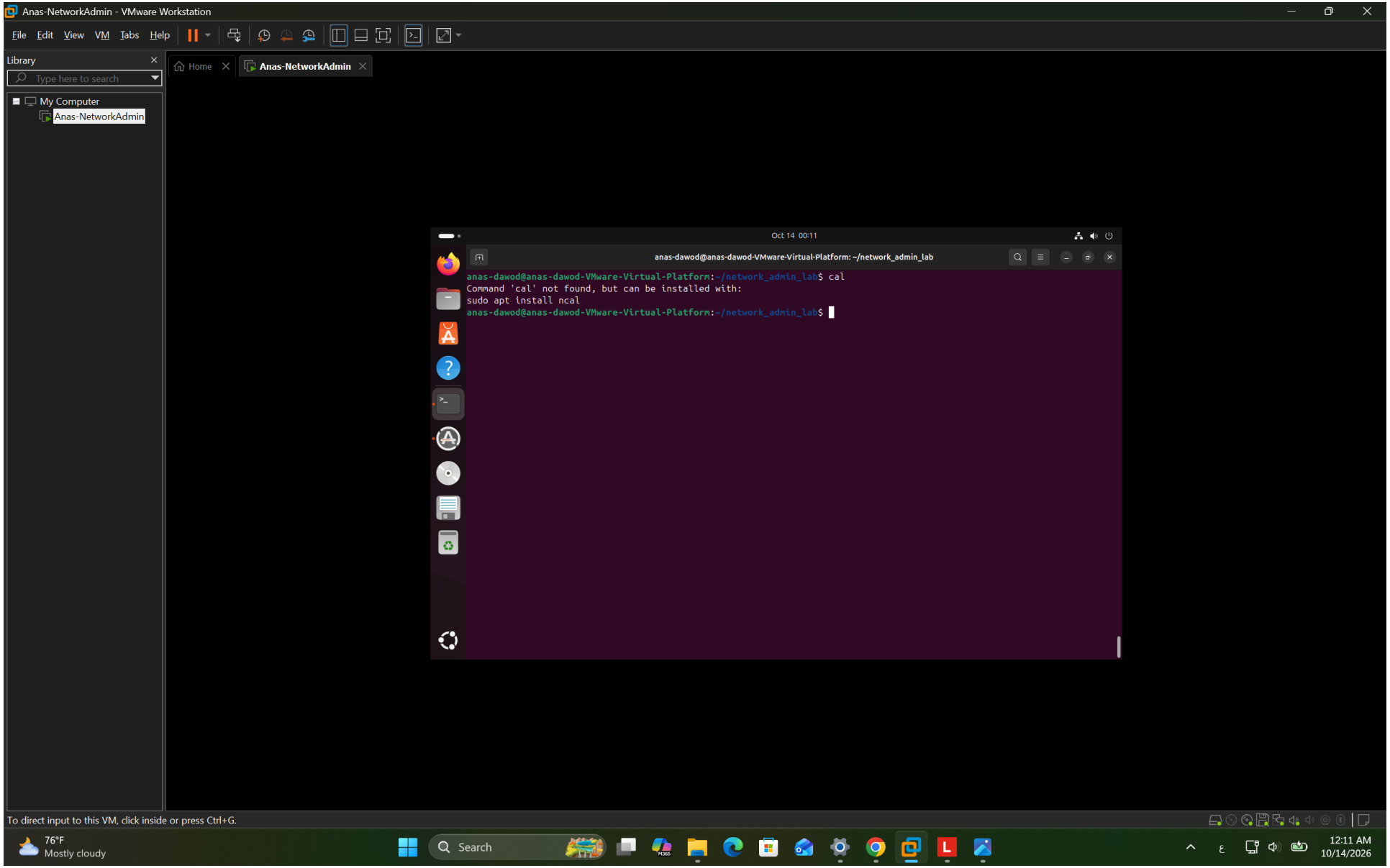
Linux Terminal Evidence 30 / 37

Screenshot 2026-10-14 000642



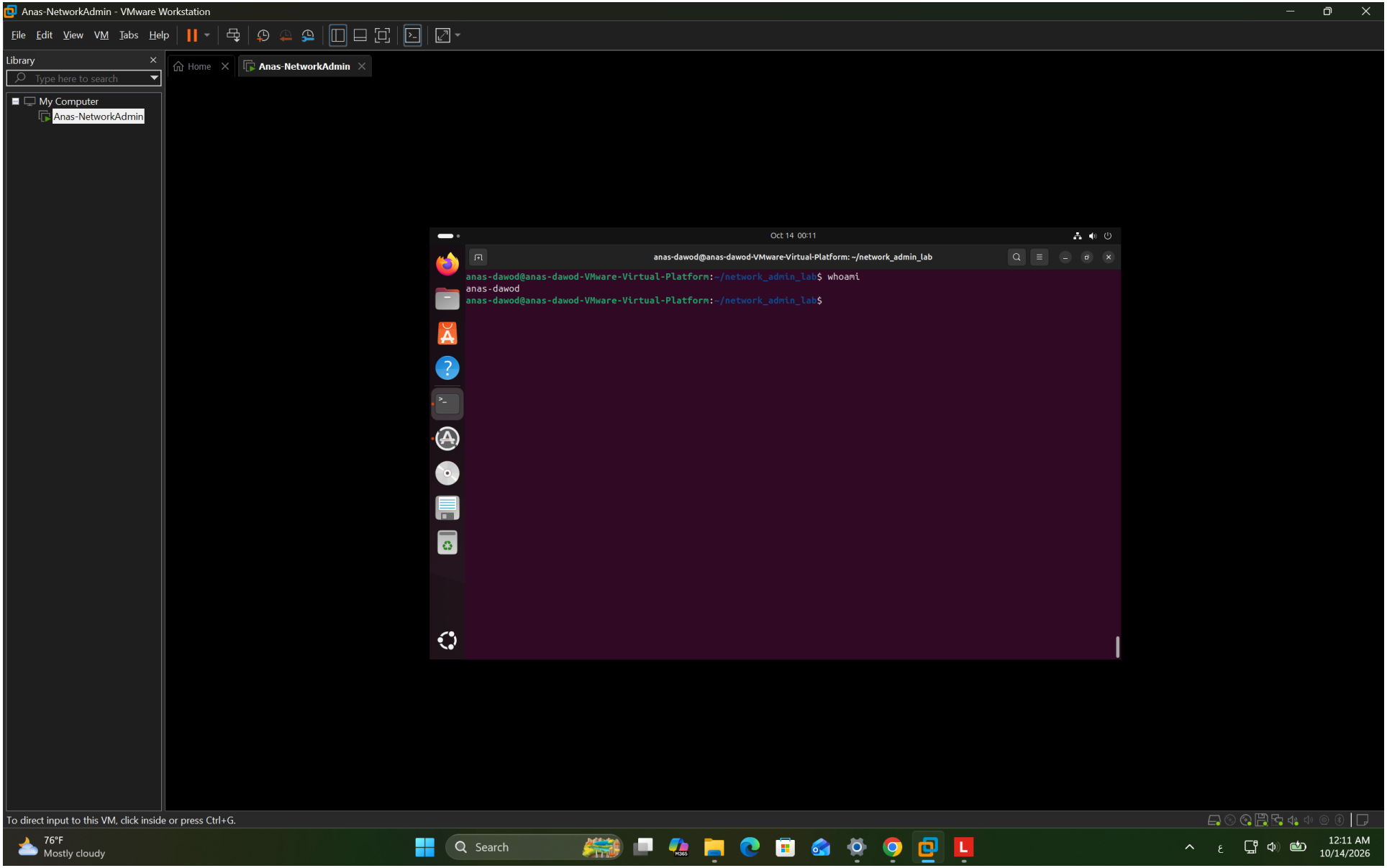
Linux Terminal Evidence 31 / 37

Screenshot 2026-10-14 001125



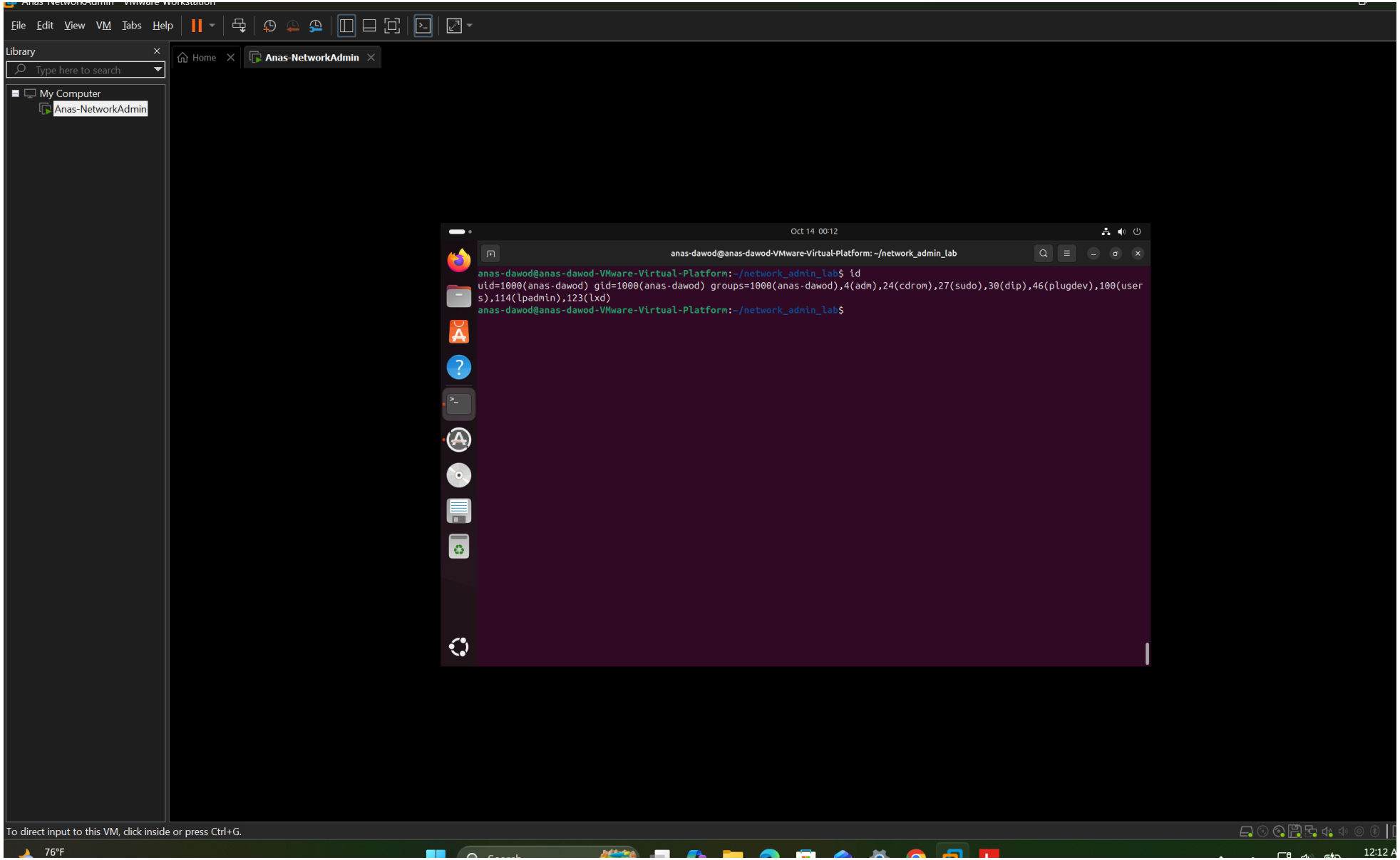
Linux Terminal Evidence 32 / 37

Screenshot 2026-10-14 001154



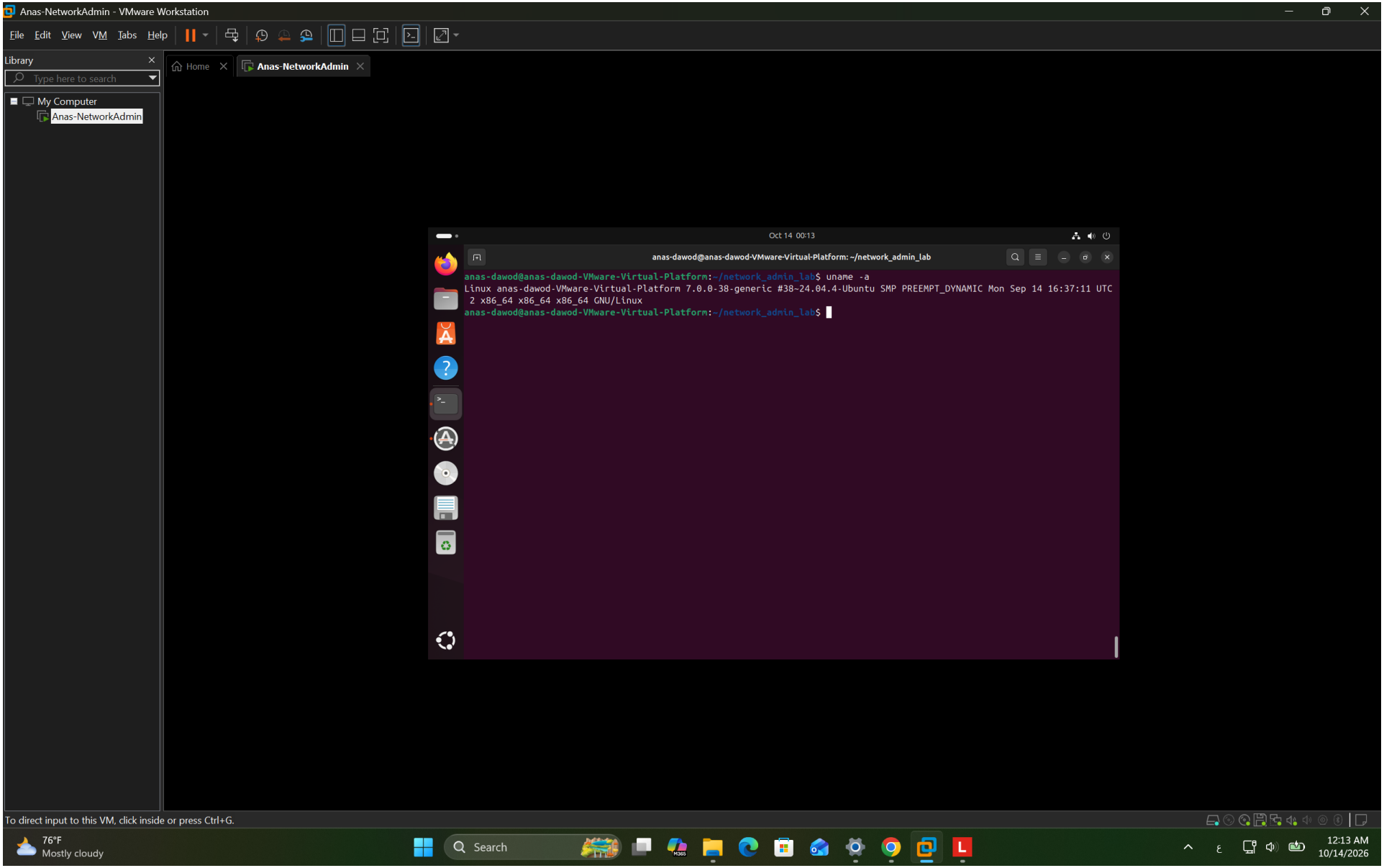
Linux Terminal Evidence 33 / 37

Screenshot 2026-10-14 001224



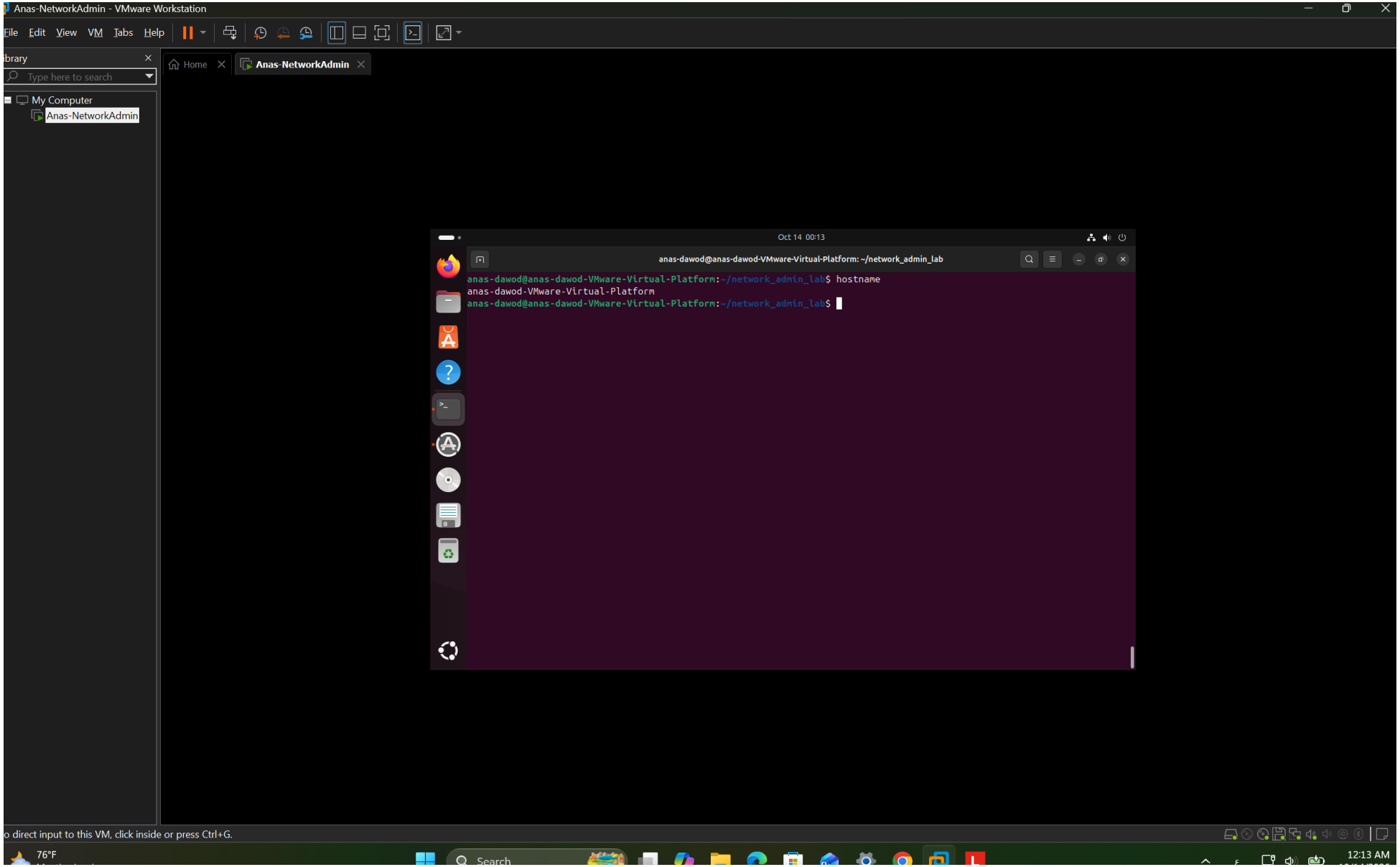
Linux Terminal Evidence 34 / 37

Screenshot 2026-10-14 001309



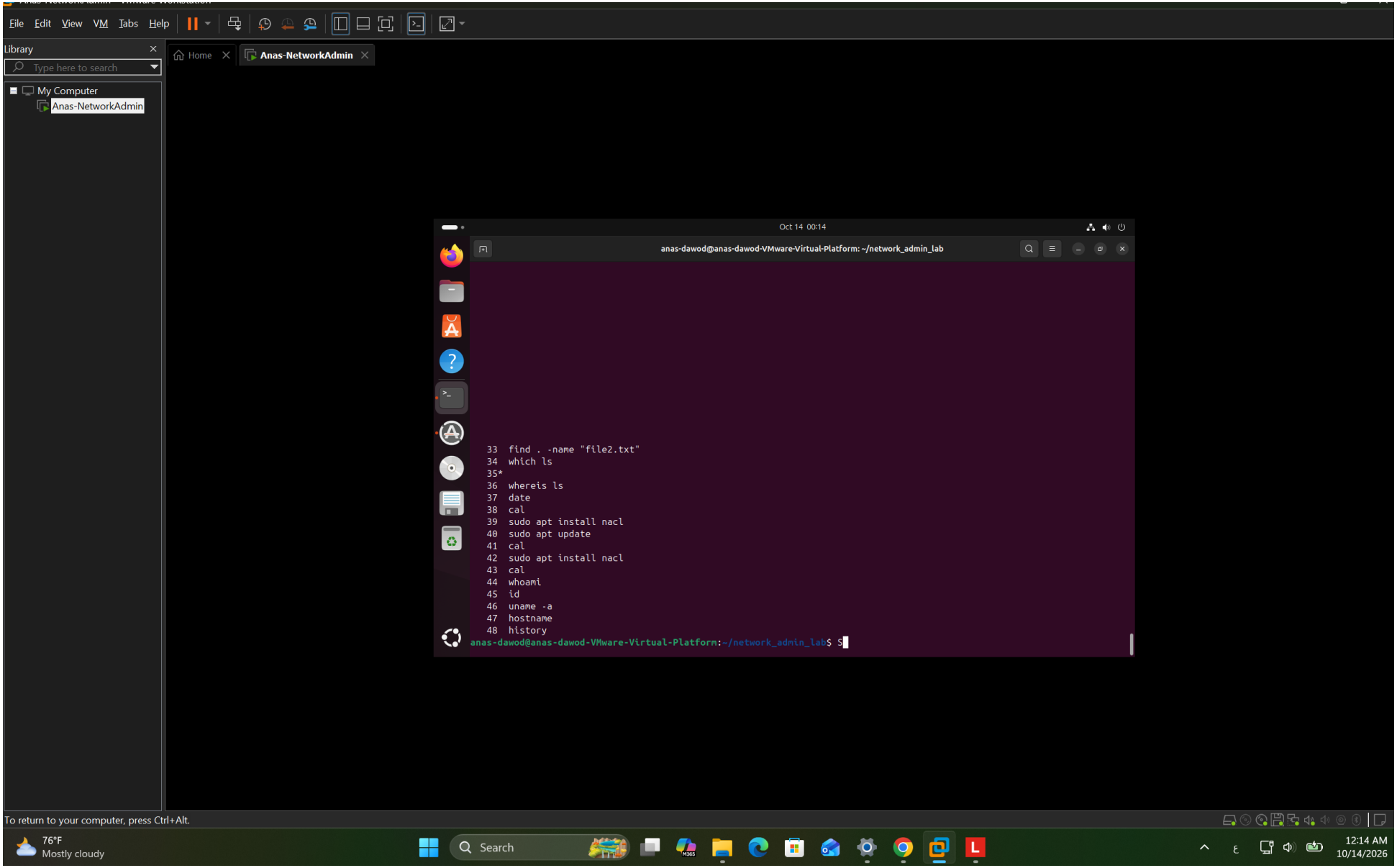
Linux Terminal Evidence 35 / 37

Screenshot 2026-10-14 001336



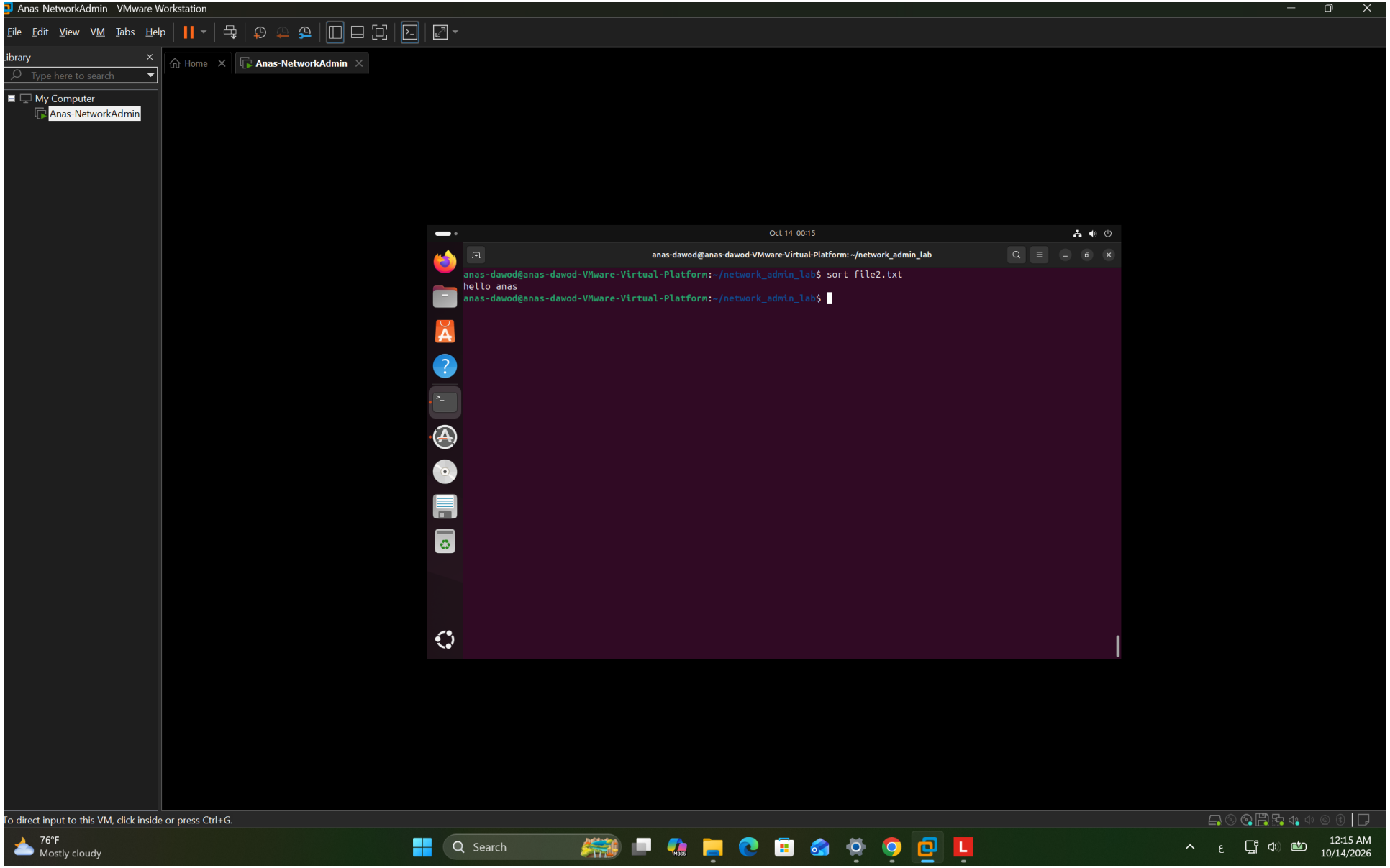
Linux Terminal Evidence 36 / 37

Screenshot 2026-10-14 001438



Linux Terminal Evidence 37 / 37

Screenshot 2026-10-14 001520



Part 2 | Network Administrator Agent

Python source file and documented execution

Script: network_agent.py (uploaded as network_agent(1).py)

Implementation: Python 3.12.3 with socket and subprocess

Checks: hostname, ping connectivity to 8.8.8.8, hostname-based IP resolution

Output: network status plus rule-based recommendations

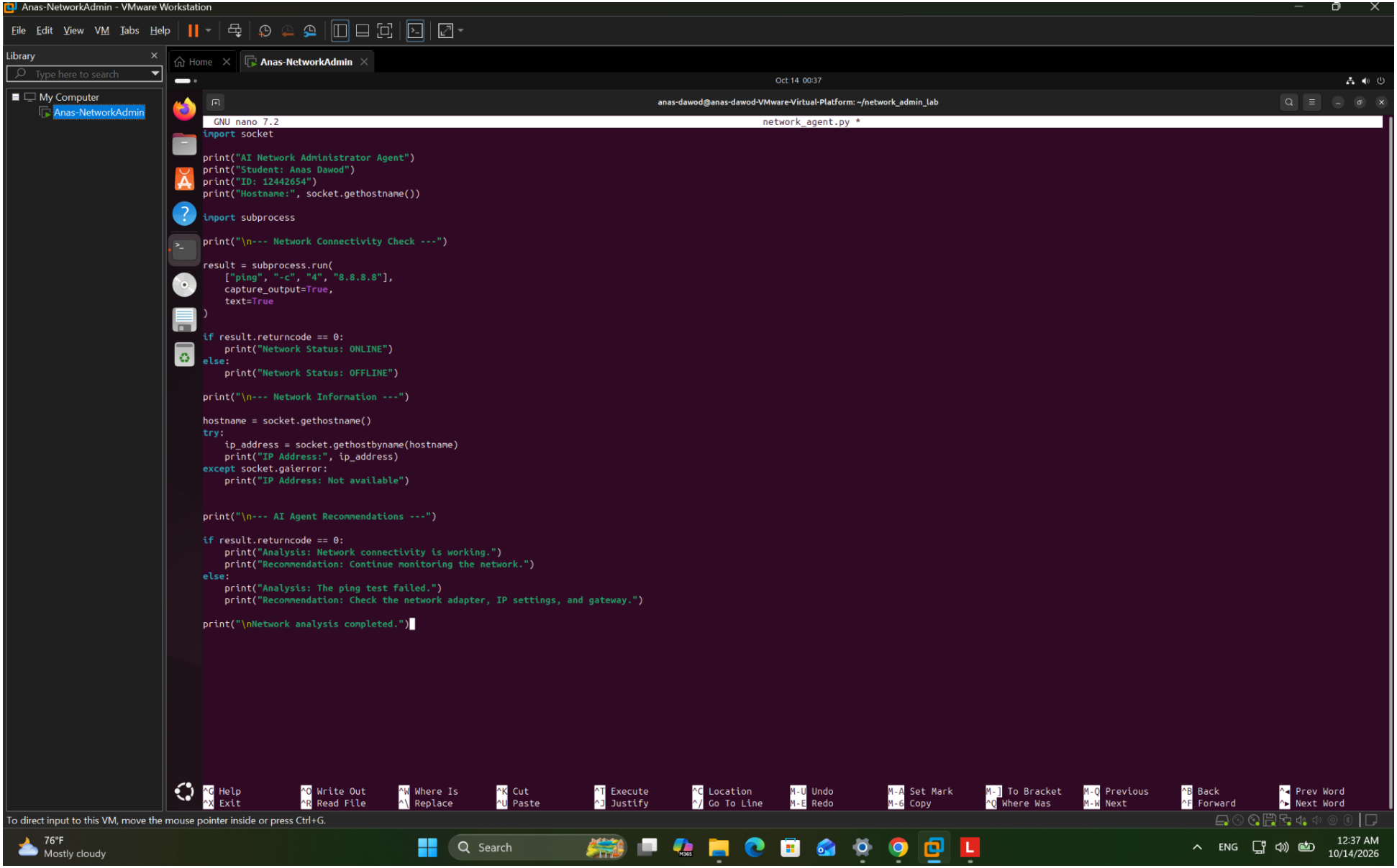
Observed run: ONLINE; IP shown 127.0.1.1 (local loopback-associated address).

Technical note: this is a rule-based diagnostic script, not a trained AI model.

A successful ping does not guarantee every Internet service is reachable.

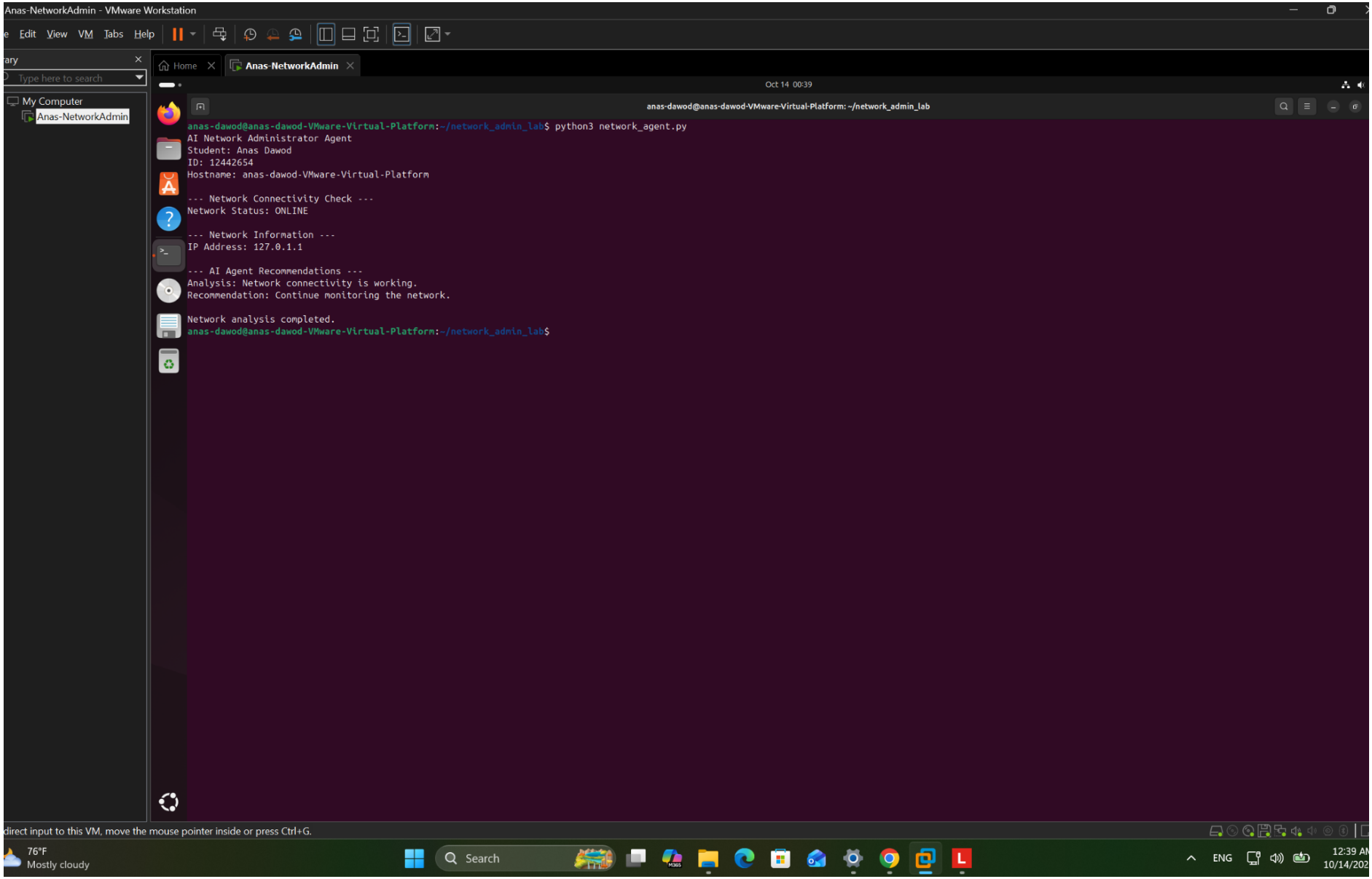
Agent | Source in Nano

Screenshot provided by student



Agent | Execution Result

Screenshot provided by student



Appendix | Python Source (1/2)

network_agent.py - exact uploaded content

```
01  import socket
02
03  print("AI Network Administrator Agent")
04  print("Student: Anas Dawod")
05  print("ID: 12442654")
06  print("Hostname:", socket.gethostname())
07
08  import subprocess
09
10  print("\n--- Network Connectivity Check ---")
11
12  result = subprocess.run(
13      ["ping", "-c", "4", "8.8.8.8"],
14      capture_output=True,
15      text=True
16  )
17
18  if result.returncode == 0:
19      print("Network Status: ONLINE")
20  else:
21      print("Network Status: OFFLINE")
22
23  print("\n--- Network Information ---")
24
25  hostname = socket.gethostname()
26  try:
27      ip_address = socket.gethostbyname(hostname)
28      print("IP Address:", ip_address)
29  except socket.gaierror:
30      print("IP Address: Not available")
31
32
33  print("\n--- AI Agent Recommendations ---")
34
35  if result.returncode == 0:
```

Appendix | Python Source (2/2)

network_agent.py - exact uploaded content

```
    print("Analysis: Network connectivity is working.")
    print("Recommendation: Continue monitoring the network.")
else:
    print("Analysis: The ping test failed.")
    print("Recommendation: Check the network adapter, IP settings, and gateway.")

print("\nNetwork analysis completed.")
```